



数据之门

博客园

首页

新随笔

联系

订阅

管理

【锁】Oracle死锁（DeadLock）的分类及其模拟

【锁】Oracle死锁（DeadLock）的分类及其模拟

1 BLOG文档结构图

【锁】Oracle 死锁 (DeadLock) 的分类及其模拟
1.1 BLOG 文档结构图
1.2 前言部分
1.2.1 导读和注意事项
1.2.2 本文简介
第 2 章 死锁 (DeadLock) 的分类及其模拟
2.1 死锁简介
2.1.1 什么是死锁？
2.1.2 死锁的 trace 文件
2.1.3 死锁的检测时间
2.2 死锁的分类
2.2.1 行级死锁
2.2.1.1 主键、唯一索引的死锁（会话交叉插入相同的主键值）
一、脚本
2.2.1.2 外键的死锁（外键未加索引）
一、脚本
2.2.1.3 位图(BITMAP)索引死锁
一、脚本
2.2.1.4 常见事务引发的死锁
一、模拟一：两个表之间不同顺序相互更新操作引起的死锁
二、模拟二：同一张表删除和更新之间引起的死锁
2.2.1.5 自治事务引发的死锁
一、模拟一：更新
1、脚本
二、模拟二：插入
1、脚本
2.2.2 块级死锁
2.2.2.1 ITL 简介
一、ITL 结构
二、ITL 个数
2.2.2.2 ITL 引起的死锁案例处理
2.2.2.3 ITL 死锁的模拟
About Me

2 前言部分

2.1 导读和注意事项

各位技术爱好者，看完本文后，你可以掌握如下的技能，也

- ① 死锁的概念及其trace文件
- ② 死锁的分类

公告

昵称： DB宝
园龄： 10年1个月
粉丝： 212
关注： 74
[+加关注](#)

<	2024年8月			
日	一	二	三	
28	29	30	31	
4	5	6	7	
11	12	13	14	
18	19	20	21	
25	26	27	28	
1	2	3	4	

搜索

最新随笔

- 1.最新博客地址：www.dbbae.com
- 2.小麦苗GreenPlum学习
- 3.【PG】小麦苗PGCA+P 寄
- 4.DB2数据库生成html巡
- 5.纯SQL生成openGauss! 检报告
- 6.TiDB数据库生成html格
- 7.达梦数据库使用disql生 检报告

- ③ 行级死锁的模拟
- ④ ITL的概念、ITL结构
- ⑤ ITL引发的死锁处理
- ⑥ ITL死锁的模拟

Tips:

- ① 本文在itpub (<http://blog.itpub.net/26736162>)、博客园 (<http://www.cnblogs.com/lhrbest>)和微信公众号 (xiaomaimiaolhr) 上有同步更新。
- ② 文章中用到的所有代码、相关软件、相关资料及本文的pdf版本都请前往小麦苗的云盘下载，小麦苗的云盘地址见：<http://blog.itpub.net/26736162/viewspace-1624453/>。
- ③ 若网页文章代码格式有错乱，请尝试以下办法：①去博客园地址阅读，②下载pdf格式的文档来阅读。
- ④ 在本篇BLOG中，代码输出部分一般放在一行一列的表格中。其中，需要特别关注的地方我都用灰色背景和粉红色字体来表示，比如在下边的例子中，thread 1的最大归档日志号为33，thread 2的最大归档日志号为43是需要特别关注的地方；而命令一般使用黄色背景和红色字体标注；对代码或代码输出部分的注释一般采用蓝色字体表示。

```
List of Archived Logs in backup set 11

Thrd Seq      Low SCN      Low Time      Next SCN      Next Time
-----
1      32          1621589      2015-05-29 11:09:52 1625242      2015-05-29 11:15:48
1      33          1625242      2015-05-29 11:15:48 1625293      2015-05-29 11:15:58
2      42          1613951      2015-05-29 10:41:18 1625245      2015-05-29 11:15:49
2      43          1625245      2015-05-29 11:15:49 1625253      2015-05-29 11:15:53

[ZHLHRDB1:root]:/>lsvg -o
T_XLHRD_APP1_vg
rootvg

[ZHLHRDB1:root]:/>
00:27:22 SQL> alter tablespace idxtbs read write;
====> 2097152*512/1024/1024/1024=1G
```

本文如有错误或不完善的地方请大家多多指正，ITPUB留言或QQ皆可，您的批评指正是我写作的最大动力。

2.2 本文简介

写了近大半年的书了，碰到了各种困难，不过幸运的是基本上都——克服了。前段时间工作上碰到一个很奇怪的死锁问题，由业务发出来的SQL来看是不太可能产生死锁的，不过的的确确实实在在的产生了，那作者是碰到了哪一类的死锁呢？ITL死锁！！有关当时的案例可以参考：<http://blog.itpub.net/26736162/viewspace-2124771/>和<http://blog.itpub.net/26736162/viewspace-2124735/>。于是，作者就把死锁可能出现的情况都分类总结了一下，分享给大家，欢迎大家指出错误。本文内容也将写入作者的新书中，欢迎大家提前订阅。

第二章 死锁（DeadLock）的分类及其模拟

1 死锁简介

1.1 什么是死锁？

所谓死锁，是指两个或两个以上的进程在执行过程中，因争夺资源而造成的一种互相等待的现象，若无外力作用，它们都将无法推进下去。此时称系统处于死锁状态或系统产生了死锁，这些永远在互相等待的进程称为死锁进程。Oracle对于“死锁”是要做处理的，而不是不闻不问。

```
[ZFLHRDB1:oracle]:/oracle>oerr ora 60
00060, 00000, "deadlock detected while waiting for resource"
// *Cause: Transactions deadlocked one another while waiting for resources.
// *Action: Look at the trace file to see the transactions and resources
//          involved. Retry if necessary.
```

Cause: Your session and another session are waiting for are source locked by the other. This condition is known AS a deadlock. To resolve the deadlock, one or more statements were rolled back for the other session to continue work.

Action Either: 1. Enter arollback statement and re-execute all statements since the last commit or 2. Wait until the lock is released, possibly a few minutes, and then re-execute the rolled back statements.

1.2 死锁的trace文件

Oracle中产生死锁的时候会在alert告警日志文件中记录死锁且当发生死锁时都会生成一个trace文件，这个文件名在alert\$Daemon) 进程统一管理各个节点之间的锁资源的，所以，RAC环境在RAC环境中，告警日志的形式如下所示：

在RAC环境都有Deadlock这个关键词，而环境中，是由LMD (Lock Manager)来生成的。

8.在VMware Workstation
Oracle 21c rac集群环境

9.我的最新博客地址: <http://p.com/>

10.我的新博客地址<https://>

积分与排名

积分 - 586759

排名 - 1003

随笔档案 (183)

2023年9月(2)

2022年11月(1)

2021年10月(4)

2021年8月(1)

2021年6月(2)

2021年5月(2)

2021年4月(4)

2021年3月(5)

2021年2月(3)

2021年1月(1)

2019年6月(1)

2019年5月(1)

2019年3月(2)

2019年2月(2)

2019年1月(1)

2018年11月(1)

2018年8月(4)

2018年7月(4)

```
Mon Jun 20 10:10:56 2016

Global Enqueue Services Deadlock detected. More info in file

/u01/app/oracle/diag/rdbms/rac1hr/rac1hr2/trace/rac1hr2_lmd0_19923170.trc.
```

在单机环境中，告警日志的形式如下所示：

```
Mon Jun 20 12:10:56 2016

ORA-00060: Deadlock detected. More info in file

/u01/app/oracle/diag/rdbms/dlhr/dlhr/trace/dlhr_ora_16973880.trc.
```

通常来讲，对于单机环境，当有死锁发生后，在trace文件中会看到如下的日志信息：

```
[Transaction Deadlock]

The following deadlock is not an ORACLE error. It is a
deadlock due to user error in the design of an application
or from issuing incorrect ad-hoc SQL. The following
information may aid in determining the deadlock:

Deadlock graph:
-----Blocker(s)-----
Resource Name      process session holds waits
TX-000d0005-00000047 41      37      X
TX-0008000a-0000036b 25      34      X
-----Waiter(s)-----
Resource Name      process session holds waits
TX-000d0005-00000047 41      37      X
TX-0008000a-0000036b 25      34      X

session 37: DID 0001-0029-00000003
session 34: DID 0001-0019-0000000D
session 37: DID 0001-0029-00000003
```

图2-1 单机环境下的死锁

当看到trace文件时，需要确认一下产生锁的类型，是两行还是一行，是TX还是TM，如果只有一行那么说明是同一个SESSION，可能是自治事务引起的死锁。

对于RAC环境，当有死锁发生后，在trace文件中会看到如下的日志信息：

```
*** 2016-09-01 18:30:38.014
Submitting asynchronous dump request [28]. summary=[ges process stack dump (kjdgblkrdml)].
Global blockers dump end:
Global Wait-For-Graph(WFG) at ddTS[0.2fe0] :
BLOCKED 0x7000008e9c8bc28 3 wq 2 cvtops x1 TX 0x159001e.0x2379(ext 0x5,0x0) [1006-0065-0019365C] inst 1
BLOCKER 0x7000008e9ab4b28 3 wq 1 cvtops x28 TX 0x159001e.0x2379(ext 0x5,0x0) [2005-005E-00185E15] inst 2
BLOCKED 0x700000891b48708 3 wq 2 cvtops x1 TX 0x1c2001d.0x4b82(ext 0x2,0x0) [2005-005E-00185E15] inst 2
BLOCKER 0x7000008e9c8b148 3 wq 1 cvtops x28 TX 0x1c2001d.0x4b82(ext 0x2,0x0) [1004-004D-0000C03E] inst 1
BLOCKED 0x70000089a636970 3 wq 2 cvtops x1 TX 0x1c0000b.0x18f6(ext 0x2,0x0) [1004-004D-0000C03E] inst 1
BLOCKER 0x7000008e9c8b4e8 3 wq 1 cvtops x28 TX 0x1c0000b.0x18f6(ext 0x2,0x0) [1005-0058-000DD3D9] inst 1
BLOCKED 0x700000891d5fc50 3 wq 2 cvtops x1 TX 0xee0009.0x5ea6(ext 0x2,0x0) [1005-0058-000DD3D9] inst 1
BLOCKER 0x7000008094d14e0 3 wq 1 cvtops x28 TX 0xee0009.0x5ea6(ext 0x2,0x0) [1006-0065-0019365C] inst 1
* Cancel deadlock victim lockp 0x7000008e9c8bc28
```

图2-2 RAC环境下的死锁

1.3 死锁的检测时间

死锁的监测时间是由隐含参数`_lm_dd_interval`来控制的，在Oracle 11g中，隐含参数`_lm_dd_interval`的值默认为10，而在Oracle 10g中该参数默认为60，单位为秒。

```
SYS@oraLHRDB2> SELECT A.INDX,
2      A.KSPPINM NAME,
3      A.KSPDESC,
4      B.KSPSTVL
5 FROM   X$KSPPI A,
6        X$KSPPCV B
7 WHERE  A.INDX = B.INDX
8 AND LOWER(A.KSPPINM) LIKE LOWER('%&PARAMETER%');

ENTER VALUE FOR PARAMETER: _lm_dd_interval

OLD 8: AND LOWER(A.KSPPINM) LIKE LOWER('%&PARAMETER%')
NEW 8: AND LOWER(A.KSPPINM) LIKE LOWER('%_LM_DD_INTERVAL%')

      INDX NAME                                KSPDESC                                KSPSTVL
-----
578 _lm_dd_interval      dd time interval in seconds      10
```

可以看到该隐含参数的值为10。

2 死锁的分类

有人的地方就有江湖，有资源阻塞的地方就可能死锁。Oracle中最常见的死锁分为：行级死锁（Row-Level Deadlock）和块级死锁（Block-Level Deadlock），其中，行级死锁分为①主键、唯一索引的死锁（会话交叉插入相同的主键值），②外键未加索引，③表上的位图索引遭到并发更新，④常见事务引发的死锁（相互更新操作引起的死锁；同一张表删除和insert操作引起的死锁），⑤自治事务引发的死锁。块级死锁主要指（Transaction List）死锁。

死锁分类图如下所示：

2018年6月(1)

2018年5月(1)

2018年4月(4)

2018年3月(2)

2018年2月(3)

2018年1月(1)

2017年11月(2)

2017年9月(1)

2017年8月(1)

2017年7月(10)

2017年6月(3)

2017年5月(1)

2017年4月(2)

2017年3月(6)

2017年2月(2)

2017年1月(6)

2016年12月(8)

2016年11月(10)

2016年10月(7)

2016年9月(11)

2016年8月(7)

2016年6月(1)

更多

文章档案 (73)

2016年10月(1)

2016年9月(7)

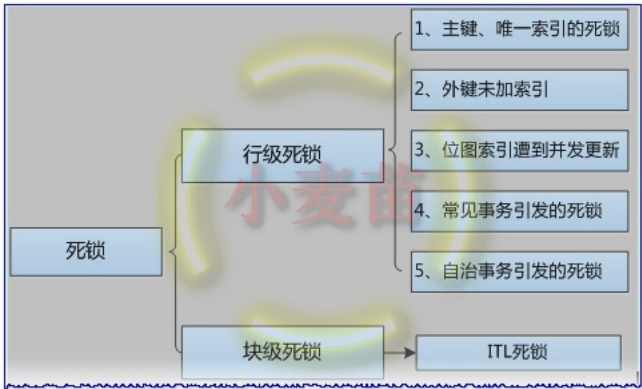
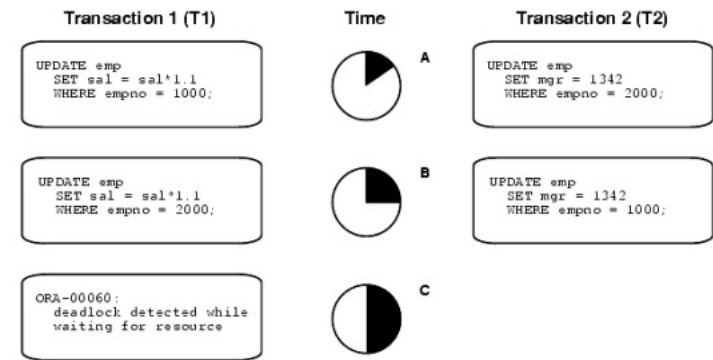


图2-3 死锁的分类图

2.1 行级死锁

行级锁的发生如下图所示，在A时间，TRANSACTION1和TRANSACTION2分别锁住了它们要UPDATE的一行数据，没有任何问题。但每个TRANSACTION都没有终止。接下来在B时间，它们又试图UPDATE当前正被对方TRANSACTION锁住的行，因此双方都无法获得资源，此时就出现了死锁。之所以称之为死锁，是因为无论每个TRANSACTION等待多久，这种锁都不会被释放。



行级锁的死锁一般是由于应用逻辑设计的问题造成的，其解决方法是通过分析trace文件定位出造成死锁的SQL语句、被互相锁住资源的对象及其记录等信息，提供给应用开发人员进行分析，并修改特定或一系列表的更新（UPDATE）顺序。
以下模拟各种行级死锁的产生过程，版本都是11.2.0.4。

2.1.1 主键、唯一索引的死锁（会话交叉插入相同的主键值）

主键的死锁其本质是唯一索引引起的死锁，这个很容易模拟出来的，新建一张表，设置主键（或创建唯一索引）后插入一个值，然后不要COMMIT，另一个会话插入另一个值，也不要COMMIT，然后再把这两个插入的值互相交换一下，在两个会话中分别插入，死锁就会产生。

会话1, sid为156:

```
SYS@lhrdb> set sqlprompt "_user'@'_connect_identifier S1> "
SYS@lhrdb S1> DROP TABLE T_DEADLOCK_PRIMARY_LHR;

Table dropped.

====>>>> CREATE UNIQUE INDEX IDX_TEST_LHR ON T_DEADLOCK_PRIMARY_LHR(ID);
SYS@lhrdb S1> CREATE TABLE T_DEADLOCK_PRIMARY_LHR(ID NUMBER PRIMARY KEY);

Table created.

SYS@lhrdb S1> select userenv('sid') from dual;

USERENV('SID')
-----
156

SYS@lhrdb S1> INSERT INTO T_DEADLOCK_PRIMARY_LHR VALUES(1);

1 row created.
```

2016年8月(8)

2016年7月(6)

2016年6月(5)

2016年5月(4)

2016年4月(30)

2016年3月(5)

2016年2月(4)

2015年5月(3)

阅读排行榜

- 1. Oracle 19C的下载和安装和静默安装) (79460)
- 2. 【故障处理】ORA-28003: authentication protocol error (38823)
- 3. 如何更改图片的背景色星) (38823)
- 4. 【方法】Oracle用户密码的登陆问题(30649)
- 5. 【RAC】RAC相关基础知识(27241)
- 6. 【故障处理】分布式事务解决(27241)
- 7. 【等待事件】等待事件 queue(队列等待)(25711)
- 8. PLSQL Developer软件
- 9. 【故障解决】OGG-004158)
- 10. 【故障处理】imp-0022157)
- 11. 【故障解决】ORA-004488)
- 12. 【OGG】OGG基础知识
- 13. 【知识点整理】Oracle G、APPEND、ARCHIVE

SYS@lhrdb S1>

会话2, sid为156:

```
SYS@lhrdb> set sqlprompt "_user'@'_connect_identifier S2> "
SYS@lhrdb S2> select userenv('sid') from dual;

USERENV('SID')
-----
191

SYS@lhrdb S2> INSERT INTO T_DEADLOCK_PRIMARY_LHR VALUES(2);

1 row created.

SYS@lhrdb S2> INSERT INTO T_DEADLOCK_PRIMARY_LHR VALUES(1);
```

===== >>>> 产生了阻塞

```
SYS@lhrdb S2> INSERT INTO T_DEADLOCK_PRIMARY_LHR VALUES(1);
```

```
SELECT NVL(A.SQL_ID, A.PREV_SQL_ID) SQL_ID,
       A.BLOCKING_SESSION,
       A.SID,
       A.SERIAL#,
       A.LOGON_TIME,
       A.EVENT
FROM   GV$SESSION A
WHERE  A.SID IN (156,191)
ORDER BY A.LOGON_TIME;
```

SQL_ID	BLOCKING_SESSION	SID	SERIAL#	LOGON_TIME	EVENT
1 acam7jgsbxqx3	156	156	5	2016/9/22 9:10:28	SQL*Net message from client ...
2 acam7jgsbxqx3	156	191	313	2016/9/22 15:44:32	enq: TX - row lock contention ...

156阻塞了191会话, 即会话1阻塞了会话2。
会话1再次插入数据:

```
SYS@lhrdb S1> INSERT INTO T_DEADLOCK_PRIMARY_LHR VALUES(2);
```

===== >>>> 产生了阻塞

此时, 去会话2看的时候, 已经报出了死锁的错误:

```
SYS@lhrdb S2> INSERT INTO T_DEADLOCK_PRIMARY_LHR VALUES(1);

INSERT INTO T_DEADLOCK_PRIMARY_LHR VALUES(1)
*
ERROR at line 1:
ORA-00060: deadlock detected while waiting for resource
```

此时的阻塞已经发生了变化:

SQL_ID	BLOCKING_SESSION	SID	SERIAL#	LOGON_TIME	EVENT
1 6kjn6k13dpuzh	191	156	5	2016/9/22 9:10:28	enq: TX - row lock contention ...
2 acam7jgsbxqx3		191	313	2016/9/22 15:44:32	SQL*Net message from client ...

告警日志:

Fri Sep 23 09:03:11 2016

REDO、UNDO和执行速

14. 【OS】NMON的简介

15. 【OGG】OGG的下载

16. 【故障•监听】TNS-1
17和 Linux Error: 32: E
60)

17. Oracle 12cR1 RAC 在
tation上安装(上)—OS环

18. 【故障处理】队列等
ITL entry引起的死锁处理

19. 【故障处理】DBCA
6(17184)

20. 【函数】wm_concat

21. 【数据泵】EXPDP导

22. 【体系结构】Oracle

23. 【OGG】OGG的单向
DL(二)(15811)

24. 【故障解决】enq: PS
577)

25. 【RAC】rac环境下的
原(14920)

26. 数据库笔试面试题
L等) (13642)

27. 【OGG】OGG简单配
3607)

28. 【方法】如何限定IP
(13283)

29. AIX 静默安装11gR2 I

30. 【OEM】OEM安装维

31. 【书评:Oracle查询优
三章(13026)

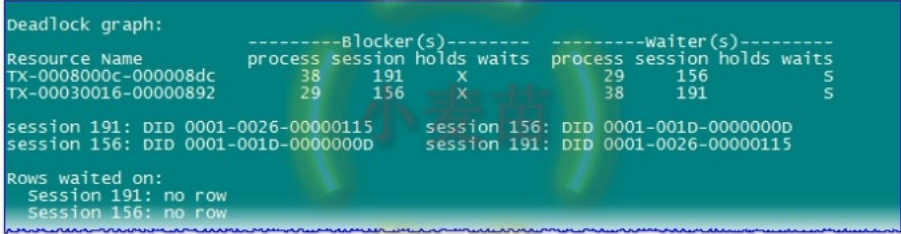
32. AIX安装单实例11gR2
6)

ORA-00060: Deadlock detected. More info in file
/oracle/app/oracle/diag/rdbms/lhrdb/lhrdb/trace/lhrdb_ora_25886852.trc.

其内容可以看到很经典的一段：

Deadlock graph:

```
-----Blocker(s)-----      -----Waiter(s)-----  
-----  
Resource Name      process session holds waits  process session  
holds waits  
TX-0008000c-000008dc      38      191      X      29  
156      S  
TX-00030016-00000892      29      156      X      38  
191      S  
  
session 191: DID 0001-0026-00000115      session 156: DID 0001-001D-0000000D  
session 156: DID 0001-001D-0000000D      session 191: DID 0001-0026-00000115  
  
Rows waited on:  
Session 191: no row  
Session 156: no row
```



Deadlock graph:

```
-----Blocker(s)-----      -----waiter(s)-----  
Resource Name      process session holds waits  process session holds waits  
TX-0008000c-000008dc      38      191      X      29      156      S  
TX-00030016-00000892      29      156      X      38      191      S  
  
session 191: DID 0001-0026-00000115      session 156: DID 0001-001D-0000000D  
session 156: DID 0001-001D-0000000D      session 191: DID 0001-0026-00000115  
  
Rows waited on:  
Session 191: no row  
Session 156: no row
```

这就是主键的死锁，模拟完毕。
此时，若是会话2执行提交后，会话1就会报错，违反唯一约束：

```
SYS@lhrdb S1> INSERT INTO T_DEADLOCK_PRIMARY_LHR VALUES(2);  
  
INSERT INTO T_DEADLOCK_PRIMARY_LHR VALUES(2)  
*  
ERROR at line 1:  
ORA-00001: unique constraint (SYS.SYS_C0011517) violated
```

一、脚本

会话1:

```
DROP TABLE T_DEADLOCK_PRIMARY_LHR;  
CREATE TABLE T_DEADLOCK_PRIMARY_LHR(ID NUMBER PRIMARY KEY);  
--CREATE UNIQUE INDEX IDX_TEST_LHR ON T_DEADLOCK_PRIMARY_LHR(ID);  
select userenv('sid') from dual;  
INSERT INTO T_DEADLOCK_PRIMARY_LHR VALUES(1);
```

会话2:

```
select userenv('sid') from dual;  
INSERT INTO T_DEADLOCK_PRIMARY_LHR VALUES(2);  
INSERT INTO T_DEADLOCK_PRIMARY_LHR VALUES(1);
```

会话1: ---死锁产生

```
INSERT INTO T_DEADLOCK_PRIMARY_LHR VALUES(2);
```

33. oracle中如何判断一个汉字(12161)

34. Oracle恢复流程图(11

35. 【RAC】RAC For W
装过程中碰到的问题(九)(

36. 【OGG】OGG的单向
(一)(11816)

37. 使用OpenFiler来模拟
SM共享盘及多路径 (mu
1794)

38. 【等待事件】等待事件
system IO (控制文件) +
3)

39. 【故障处理】队列等待
ention案例(11548)

40. 【云和墨墨】性能优化
合理配置大内存页(Hugef

评论排行榜

1. Oracle 19C的下载和安
装和静默安装) (8)

2. 【故障处理】队列等待
ntion案例(4)

3. 海量IT资料 + 各种平台
文件 + 公开课录像 + 各种

4. 【RAC】RAC相关基础

5. 使用OpenFiler来模拟
M共享盘及多路径 (mult

6. 【体系结构】Oracle参

7. 【DB宝47】企业知识
器之Confluence(2)

8. 【RMAN】Oracle中如
件? 备份控制文件的方式

9. 【DG】利用闪回数据库
复Failover后的DG环境(2

```
SELECT NVL(A.SQL_ID, A.PREV_SQL_ID) SQL_ID,
       A.BLOCKING_SESSION,
       A.SID,
       A.SERIAL#,
       A.LOGON_TIME,
       A.EVENT
FROM GV$SESSION A
WHERE A.SID IN (156,191)
ORDER BY A.LOGON_TIME;
```

2.1.2 外键的死锁（外键未加索引）

外键未加索引很容易导致死锁。在以下两种情况下，Oracle在修改父表后会对子表加一个全表锁：

1. 如果更新了父表的主键，由于外键上没有索引，所以子表会被锁住。
2. 如果删除了父表中的一行，由于外键上没有索引，整个子表也会被锁住。

总之，就是**更新或者删除父表的主键**，都会导致对其子表加一个全表锁。

如果父表存在删除记录或者更改外键列的情形，那么就需要在子表上为外键列创建索引。

除了全表锁外，在以下情况下，未加索引的外键也可能带来问题：

1. 如果有ON DELETE CASCADE，而且没有对子表加索引：例如，EMP是DEPT的子表，DELETE DEPTNO = 10应该CASCADE（级联）至EMP[4]。如果EMP中的DEPTNO没有索引，那么删除DEPT表中的每一行时都会对EMP做一个全表扫描。这个全表扫描可能是不必要的，而且如果从父表删除多行，父表中每删除一行就要扫描一次子表。
2. 从父表查询子表：再次考虑EMP/DEPT例子。利用DEPTNO查询EMP表是相当常见的。如果频繁地运行以下查询（例如，生成一个报告），你会发现没有索引会使查询速度变慢：
SELECT * FROM DEPT, EMP
WHERE EMP.DEPTNO = DEPT.DEPTNO
AND DEPT.DEPTNO = :X;

那么，什么时候不需要对外键加索引呢？答案是，一般来说，当满足以下条件时不需要加索引：

1. 没有从父表删除行。
2. 没有更新父表的惟一键/主键值。
3. 没有从父表联结子表。

如果满足上述全部3个条件，那你完全可以跳过索引，不需要对外键加索引。不过个人还是强烈建议对子表添加索引，既然已经创建了外键，就不在乎再多一个索引吧，因为一个索引所增加的代价比如死锁，与缺失这个索引所带来的问题相比，是微不足道的。

子表上为外键列建立索引，可以：

- 1) 提高针对外键列的查询或改动性能
- 2) 减小表级锁粒度，降低死锁发生的可能性

外键的死锁可以这样通俗的理解：有两个表A和B：A是父表，B是子表。如果没有在B表中的外键加上索引，那么A表在更新或者删除主键时，都会在表B上加一个全表锁。这是为什么呢？因为我们没有给外键加索引，在更新或者删除A表主键的时候，需要查看子表B中是否有对应的记录，以判断是否可以更新删除。那如何查找呢？当然只能在子表B中一条一条找了，因为我们没有加索引吗。既然要在子表B中一条一条地找，那就得把整个子表B都锁定了。由此就会导致以上一系列问题。

实验过程：
会话1首先建立子表和父表

```
SYS@lhrdb S1> CREATE TABLE T_DEADLOCK_P_LHR (ID NUMBER PRIMARY KEY, NAME VARCHAR2(30));
--父表的主键

Table created.

SYS@lhrdb S1> INSERT INTO T_DEADLOCK_P_LHR VALUES (1, 'A');

1 row created.

SYS@lhrdb S1> INSERT INTO T_DEADLOCK_P_LHR VALUES (2, 'B');

1 row created.

SYS@lhrdb S1> CREATE TABLE T_DEADLOCK_F_LHR (FID NUMBER, FNAME VARCHAR2(30), FOREIGN
KEY (FID) REFERENCES T_DEADLOCK_P_LHR); --子表的外键

Table created.

SYS@lhrdb S1> INSERT INTO T_DEADLOCK_F_LHR VALUES (1, 'C');

1 row created.
```

10. 【BBED】BBED模拟并
2错误(2)

11. 【MySQL】Linux下M
和5.7的RPM、二进制和源

12. Oratop工具——实时
工具(2)

13. 【OS】NMON的简介

14. 【OGG】OGG基础知

15. 【函数】Oracle函数系
函数及日期函数(2)

16. 【锁】Oracle死锁（C
类及其模拟(2)

17. Oracle恢复流程图(2)

18. 【函数】wm_concat

19. 【DB宝41】监控利器
控MySQL、PG、Mongo
等(1)

20. 使用 Python 生成二

21. 【爬虫】网页抓包工
est和Response(1)

22. Oracle中如何导出存储
和触发器的定义语句？如
如何导出索引的创建语句

23. 在高并发、高负载的情
添加字段并设置DEFAULT

24. 【方法】Oracle用户
的登陆问题(1)

25. Oracle 12cR1 RAC在
tation上安装(中)——图形界

26. Oracle 12cR1 RAC在
tation上安装(上)——OS环

27. 【故障处理】ORA-28
ng authentication protc

28. 【密码】Oracle用户

```
SYS@lhrdb S1> INSERT INTO T_DEADLOCK_F_LHR VALUES (2, 'D');

1 row created.

SYS@lhrdb S1>

SYS@lhrdb S1> COMMIT;

Commit complete.
```

会话1执行一个删除操作，这时候在子表和父表上都加了一个Row-X (SX) 锁

```
SYS@lhrdb S1> delete from T_DEADLOCK_F_LHR where FID=1;

1 row deleted.

SYS@lhrdb S1> delete from T_DEADLOCK_P_LHR where id=1;

1 row deleted.
```

查询会话1的锁信息：

```
SELECT LK.SID,
       DECODE(LK.TYPE,
              'TX',
              'Transaction',
              'TM',
              'DML',
              'UL',
              'PL/SQL User Lock',
              LK.TYPE) LOCK_TYPE,
       DECODE(LK.LMODE,
              0,
              'None',
              1,
              'Null',
              2,
              'Row-S (SS)',
              3,
              'Row-X (SX)',
              4,
              'Share',
              5,
              'S/Row-X (SSX)',
              6,
              'Exclusive',
              TO_CHAR(LK.LMODE)) MODE_HELD,
       DECODE(LK.REQUEST,
              0,
              'None',
              1,
              'Null',
              2,
              'Row-S (SS)',
              3,
              'Row-X (SX)',
              4,
              'Share',
              5,
              'S/Row-X (SSX)',
              6,
              'Exclusive',
              TO_CHAR(LK.REQUEST)) MODE_REQUESTED,
       OB.OBJECT_TYPE,
       OB.OBJECT_NAME,
       LK.BLOCK,
       SE.LOCKWAIT
FROM V$LOCK LK, DBA_OBJECTS OB, V$SESSION SE
WHERE LK.TYPE IN ('TM', 'UL')
      AND LK.SID = SE.SID
      AND LK.ID1 = OB.OBJECT_ID(+)
      AND SE.SID IN (156,191)

ORDER BY SID;
```

	SID	LOCK_TYPE	MODE_HELD	MODE_REQUESTED	OBJECT_TYPE	K	LOCKWAIT
1	156	DML	Row-X (SX)	None	TABLE	0	
2	156	DML	Row-X (SX)	None	TABLE	0	

29. 【锁】Oracle锁系列(

30. 【SQL查询】集合查询

31. 【函数】Oracle函数系列
函数(1)

32. 【OEM】OEM安装维护

33. 【OGG】OGG的下载

推荐排行榜

1. 【爬虫】网页抓包工具
st和Response(4)

2. 【体系结构】Oracle参数

3. Oracle 19C的下载和安装
和静默安装) (3)

4. 【锁】Oracle锁系列(3

5. 【DB宝42】MySQL高并发
roxySQL实现读写分离和!

最新评论

1. Re: 【DB宝41】 监控利器
监控MySQL、PG、Mongo
L等
支持sqlserver吗

2. Re:如何将一个普通表复制
真不错，谢谢分享

3. Re:使用OpenFiler来模拟
中ASM共享盘及多路径（
试
openfiler2.99
Please authenticate with
s 老师这个情况怎么解决，
谢！！

4. Re:使用OpenFiler来模拟
中ASM共享盘及多路径（

BLOCK为0表示没有阻塞其它的锁。

会话2：执行另一个删除操作，发现这时候第二个删除语句等待

```
SYS@lhrdb S2> delete from T_DEADLOCK_F_LHR where FID=2;

1 row deleted.

SYS@lhrdb S2> delete from T_DEADLOCK_P_LHR where id=2;
```

===== 产生了阻塞

```
SYS@lhrdb S2> delete from T_DEADLOCK_F_LHR where FID=2;
1 row deleted.
SYS@lhrdb S2> delete from T_DEADLOCK_P_LHR where id=2;
```

SID	LOCK_TYPE	MODE_HELD	MODE_REQUESTED	OBJECT_TYPE	OBJECT_NAME	BLOCK	LOCKWAIT
1	156 DML	Row-X (SX)	None	TABLE	T_DEADLOCK_P_LHR	0	
2	156 DML	Row-X (SX)	None	TABLE	T_DEADLOCK_F_LHR	1	
3	191 DML	Row-X (SX)	None	TABLE	T_DEADLOCK_P_LHR	0	0700010061E41200
4	191 DML	Row-X (SX)	S/Row-X (SSX)	TABLE	T_DEADLOCK_F_LHR	1	0700010061E41200

BLOCK为1表示阻塞了其它的锁。

会话1执行删除语句，死锁发生

```
SYS@lhrdb S1> delete from T_DEADLOCK_P_LHR where id=1;

===== 产生了阻塞，而会话2产生了死锁
```

```
SYS@lhrdb S1>
SYS@lhrdb S1> delete from T_DEADLOCK_P_LHR where id=1;
```

```
SYS@lhrdb S2> delete from T_DEADLOCK_P_LHR where id=2;

delete from T_DEADLOCK_P_LHR where id=2
*
ERROR at line 1:
ORA-00060: deadlock detected while waiting for resource
```

SQL_ID	BLOCKING_SESSION	SID	SERIAL#	LOGON_TIME	EVENT
1 ck621pqfth3d	191	156	5	2016/9/22 9:10:28	enq: TM - contention
2 f3m0ubk40dpda	191	191	313	2016/9/22 15:44:32	SQL*Net message from client

告警日志：

试

Please authenticate with
s 老师这个情况怎么解决，
谢！！

5. Re:【爬虫】网页抓包工
quest和Response
写得非常详细，非常棒的

6. Re:【锁】Oracle锁系
真的是讲的太详细了，花
的这么完整

7. Re:【DB宝47】企业知
神器之Confluence
有点像sharepoint

8. Re:【DB宝47】企业知
神器之Confluence
本地化版已停售

9. Re:Oracle中如何导出表
包和触发器的定义语句？
构？如何导出索引的创建
赞一个，全方位的总结！

10. Re:Oracle 19C的下载
形安装和静默安装)
yum install glibc-devel
安装时报错/usr/lib64/cr
录，则需要安装glibc-de
查下。

11. Re:truncate恢复方法
哈哈 今天突然看到了，fy
实好用

```
Fri Sep 23 10:31:10 2016

ORA-00060: Deadlock detected. More info in file
/oracle/app/oracle/diag/rdbms/lhrdb/lhrdb/trace/lhrdb_ora_25886852.trc.
```

查看内容:

```
*** 2016-09-23 10:31:10.212

DEADLOCK DETECTED ( ORA-00060 )

[Transaction Deadlock]

The following deadlock is not an ORACLE error. It is a
deadlock due to user error in the design of an application
or from issuing incorrect ad-hoc SQL. The following
information may aid in determining the deadlock:

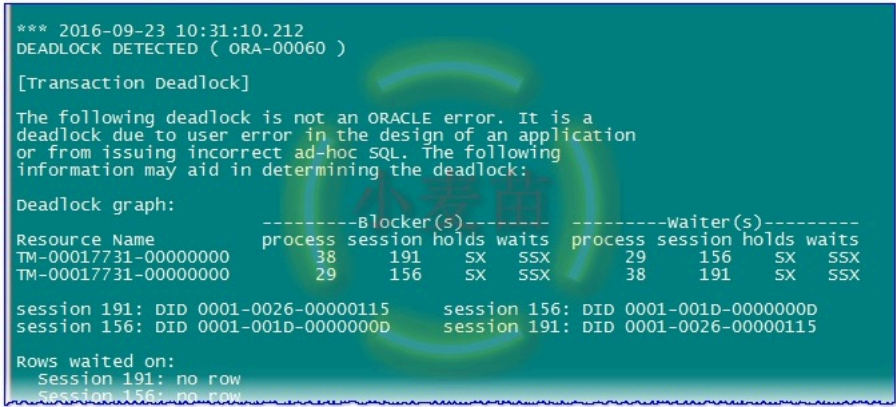
Deadlock graph:

-----Blocker(s)-----  -----Waiter(s)-----
--
Resource Name              process session holds waits  process session holds
waits
TM-00017731-00000000        38      191    SX   SSX          29      156    SX
SSX
TM-00017731-00000000        29      156    SX   SSX          38      191    SX
SSX

session 191: DID 0001-0026-00000115      session 156: DID 0001-001D-0000000D
session 156: DID 0001-001D-0000000D      session 191: DID 0001-0026-00000115

Rows waited on:

Session 191: no row
Session 156: no row
```



回滚会话建立外键列上的索引:

```
SYS@lhrdb S1> rollback;

Rollback complete.

SYS@lhrdb S1> create index ind_p_f_fid on T_DEADLOCK_F_LHR(fid);

Index created.
```

重复上面的步骤会话1删除子表记录:
---会话1:

```
SYS@lhrdb S1> delete from T_DEADLOCK_F_LHR where FID=1;
```

12. Re:使用OpenFile来精
中ASM共享盘及多路径 (

试

13. Re:Oracle 19C的下
形安装和静默安装)

6666

--规

14. Re:Oracle 19C的下
形安装和静默安装)

66666

--规

15. Re:Oracle 19C的下
形安装和静默安装)

按照图形化安装，执行sq
找到命令，重新安装了一
现19c竟然没有默认安装
竟然默认安装在源文件目

16. Re:使用 Python 生成
非常好!

17. Re:Oracle 19C的下
形安装和静默安装)

@ 亮仔的给力人生内存放

18. Re:Oracle 19C的下
形安装和静默安装)

你好，我按照图片过程安
二张图的步骤时，一直停
了。显示正在创建和启动
这是怎么回事。

19. Re:【方法】Oracle用
符时的登陆问题

老铁，sqlldr2用户密码
办法吗？如：sqlldr2 sc
@lk.com""@orcl...

1 row deleted.

SYS@lhrdb S1> delete from T_DEADLOCK_P_LHR where id=1;

1 row deleted.

SYS@lhrdb S1>

---会话2:

SYS@lhrdb S2> delete from T_DEADLOCK_F_LHR where FID=2;

1 row deleted.

SYS@lhrdb S2> delete from T_DEADLOCK_P_LHR where id=2;

1 row deleted.

所有的删除操作都可以成功执行，也没有阻塞的生成，重点就是在外键列上建立索引。

SQL_ID	BLOCKING_SESSION	SID	SERIAL#	LOGON_TIME	EVENT
1 ck621pqfth3d		156	5	2016/9/22 9:10:28	SQL*Net message from client
2 f3m0ubk40dada		191	313	2016/9/22 13:44:32	SQL*Net message from client

SID	LOCK_TYPE	MODE_HELD	MODE_REQUESTED	OBJECT_TYPE	OBJECT_NAME	BLOCK	LOCKWAIT
1	156 DML	Row-X (SX)	None	TABLE	T_DEADLOCK_P_LHR	0	
2	156 DML	Row-X (SX)	None	TABLE	T_DEADLOCK_F_LHR	0	
3	191 DML	Row-X (SX)	None	TABLE	T_DEADLOCK_P_LHR	0	
4	191 DML	Row-X (SX)	None	TABLE	T_DEADLOCK_F_LHR	0	

一、脚本

--主表

DROP TABLE T_DEADLOCK_P_LHR;

CREATE TABLE T_DEADLOCK_P_LHR (ID NUMBER PRIMARY KEY, NAME VARCHAR2(30)); --父表的主键

INSERT INTO T_DEADLOCK_P_LHR VALUES (1, 'A');

INSERT INTO T_DEADLOCK_P_LHR VALUES (2, 'B');

--子表

DROP TABLE T_DEADLOCK_F_LHR;

CREATE TABLE T_DEADLOCK_F_LHR (FID NUMBER, FNAME VARCHAR2(30), FOREIGN KEY (FID) REFERENCES T_DEADLOCK_P_LHR); --子表的外键

INSERT INTO T_DEADLOCK_F_LHR VALUES (1, 'C');

INSERT INTO T_DEADLOCK_F_LHR VALUES (2, 'D');

COMMIT;

---执行一个删除操作，这时候在子表和父表上都加了一个Row-S (SX) 锁

delete from T_DEADLOCK_F_LHR where FID=1;

delete from T_DEADLOCK_P_LHR where id=1;

---会话2: 执行另一个删除操作，发现这时候第二个删除语句等待

delete from T_DEADLOCK_F_LHR where FID=2;

delete from T_DEADLOCK_P_LHR where id=2;

---会话1: 死锁发生

delete from T_DEADLOCK_P_LHR where id=1;

---回滚会话建立外键列上的索引:

create index ind_p_f_fid on T_DEADLOCK_F_LHR(fid);

20. Re:【故障处理】ORA
ching authentication pr
有理有据，问题可复现，：

21. Re:Oracle 19C的下载
形安装和静默安装)

Oracle 19c (19.3) 可以:
下载了，也可以去Oracle
进行下载。本文是基于19

22. Re:Oracle 19C的下载
形安装和静默安装)

非常全面！多谢！伸手党

23. Re:【sessions】Orac
ocesses的大小关系 (10c
多谢分享，好文要顶。

24. Re:【RMAN】Oracle
文件？备份控制文件的方:
@ 潇湘隐者Oracle.Data
备份与恢复...

25. Re:【RMAN】Oracle
文件？备份控制文件的方:
李兄，文章中的图片来自

26. Re:在高并发、高负载
表添加字段并设置DEFAU
fcfr支持支持

27. Re:【DG】利用闪回
k) 修复Failover后的DG
真快，O(∩_∩)O哈哈~

1 row updated.

会话2：此时所有ID=2的行都被锁定

SYS@lhrdb S2> UPDATE T_DEADLOCK_BITMAP_LHR SET ID=4 WHERE ID=2 AND NAME='A';

1 row updated.

会话1：此时会话被阻塞

SYS@lhrdb S1> UPDATE T_DEADLOCK_BITMAP_LHR SET ID=4 WHERE ID=2 AND NAME='B';

====>>>> 产生了阻塞

SQL_ID	BLOCKING_SESSION	SID	SERIAL#	LOGON_TIME	EVENT
1 3kw6rdatfd0j4	191	156	5	2016/9/22 9:10:28	enq: TX - row lock contention ...
2 9dd6khsy9twir		191	313	2016/9/22 15:44:32	SQL*Net message from client ...

会话2：会话被阻塞

SYS@lhrdb S2> UPDATE T_DEADLOCK_BITMAP_LHR SET ID=3 WHERE ID=1 AND NAME='B';

====>>>> 产生了阻塞

再回到SESSION 1，发现系统检测到了死锁的发生

SYS@lhrdb S1> UPDATE T_DEADLOCK_BITMAP_LHR SET ID=4 WHERE ID=2 AND NAME='B';

UPDATE T_DEADLOCK_BITMAP_LHR SET ID=4 WHERE ID=2 AND NAME='B'

*

ERROR at line 1:

ORA-00060: deadlock detected while waiting for resource

告警日志:

Fri Sep 23 11:20:21 2016

ORA-00060: Deadlock detected. More info in file
/oracle/app/oracle/diag/rdbms/lhrdb/lhrdb/trace/lhrdb_ora_19726460.trc.

内容:

*** 2016-09-23 11:26:51.264

DEADLOCK DETECTED (ORA-00060)

[Transaction Deadlock]

The following deadlock is not an ORACLE error. It is a
deadlock due to user error in the design of an application
or from issuing incorrect ad-hoc SQL. The following
information may aid in determining the deadlock:

Deadlock graph:

-----Blocker(s)-----				-----Waiter(s)-----			
Resource Name	process	session	holds	waits	process	session	holds
TX-0009000e-00000b0f	29	156	X		38	191	S
TX-00070001-00000b2c	38	191	X				

session 156: DID 0001-001D-0000000D session 191: DID 0001-0026-00000115

36. Re:【故障处理】队列
ontention案例

然而enq: IV - contention
都没有说清楚

37. Re:【OS】NMON的

@ 潇湘隐者我是直接摘抄
整理一下，把有用的内容

38. Re:【OS】NMON的

泪奔啊，我几个月前准备
有整理好，现在你已经写
可写了！

39. Re:Oratop工具——多
控工具

@ 潇湘隐者哈哈...

40. Re:Oratop工具——多
控工具

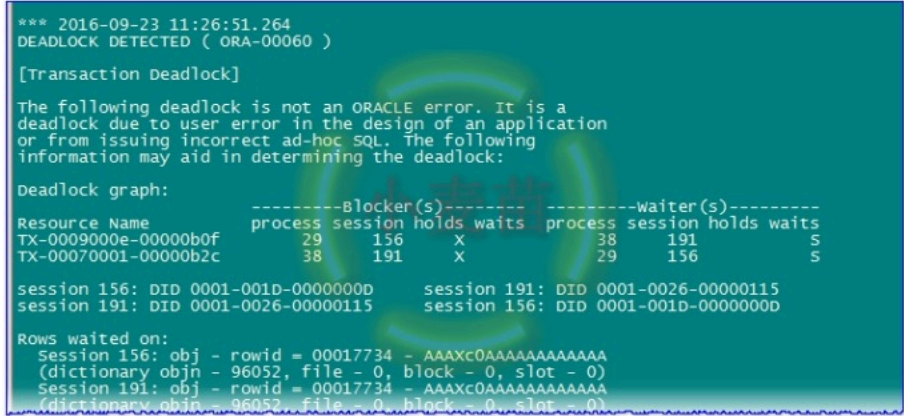
学习了，还不知道有这么

```
session 191: DID 0001-0026-00000115      session 156: DID 0001-001D-0000000D
```

Rows waited on:

```
Session 156: obj - rowid = 00017734 - AAAXc0AAAAAAAAAAAA
(dictionary objn - 96052, file - 0, block - 0, slot - 0)

Session 191: obj - rowid = 00017734 - AAAXc0AAAAAAAAAAAA
(dictionary objn - 96052, file - 0, block - 0, slot - 0)
```



死锁发生的根本原因是对于资源的排他锁定顺序不一致。上面的试验中，session1对于bitmap index中的2个位图是先锁定ID=1的位图，然后请求ID=2的位图，而在此之前ID=2的位图已经被session2锁定。session2则先锁定ID=2的位图，然后请求ID=2的位图，而此前ID=1的位图已经被session1锁定。于是，session1等待session2释放ID=2的位图上的锁，session2等待session1释放ID=1的位图上的锁，死锁就发生了

而如果我们创建的是普通的B*Tree index，重复上面的试验则不会出现任何的阻塞和死锁，这是因为锁定的只是DML操作涉及到的行，而不是所有ID相同的行。

一、脚本

```
CREATE TABLE T_DEADLOCK_BITMAP_LHR (ID NUMBER , NAME VARCHAR2(10));

INSERT INTO T_DEADLOCK_BITMAP_LHR VALUES (1,'A');

INSERT INTO T_DEADLOCK_BITMAP_LHR VALUES (1,'B');

INSERT INTO T_DEADLOCK_BITMAP_LHR VALUES (1,'C');

INSERT INTO T_DEADLOCK_BITMAP_LHR VALUES (2,'A');

INSERT INTO T_DEADLOCK_BITMAP_LHR VALUES (2,'B');

INSERT INTO T_DEADLOCK_BITMAP_LHR VALUES (2,'C');

COMMIT;

CREATE BITMAP INDEX IDX_DEADLOCK_BITMAP_LHR ON T_DEADLOCK_BITMAP_LHR (ID);

--会话1：此时所有ID=1的行都被锁定

UPDATE T_DEADLOCK_BITMAP_LHR SET ID=3 WHERE ID=1 AND NAME='A';

--会话2：此时所有ID=2的行都被锁定

UPDATE T_DEADLOCK_BITMAP_LHR SET ID=4 WHERE ID=2 AND NAME='A';

--会话1：此时会话被阻塞

UPDATE T_DEADLOCK_BITMAP_LHR SET ID=4 WHERE ID=2 AND NAME='B';

--会话2：会话被阻塞，再回到SESSION 1，发现系统检测到了死锁的发生

UPDATE T_DEADLOCK_BITMAP_LHR SET ID=3 WHERE ID=1 AND NAME='B';

--会话1：

UPDATE T_DEADLOCK_BITMAP_LHR SET ID=4 WHERE ID=2 AND NAME='B';

UPDATE T_DEADLOCK_BITMAP_LHR SET ID=4 WHERE ID=2 AND NAME='B'

*

ERROR at line 1:
```

ORA-00060: deadlock detected while waiting for resource

2.1.4 常见事务引发的死锁

如果你有两个会话，每个会话都持有另一个会话想要的资源，此时就会出现死锁（deadlock）。例如，如果我的数据库中有两个表A和B，每个表中都只有一行，就可以很容易地展示什么是死锁。我要做的只是打开两个会话（例如，两个SQL*Plus会话）。在会话A中更新表A，并在会话B中更新表B。现在，如果我想在会话B中更新表A，就会阻塞。会话A已经锁定了这一行。这不是死锁；只是阻塞而已。因为会话A还有机会提交或回滚，这样会话B就能继续。如果我再回到会话A，试图更新表B，这就会导致一个死锁。要在这两个会话中选择一个作为“牺牲品”，让它的语句回滚。

想要更新表B的会话A还阻塞着，Oracle不会回滚整个事务。只会回滚与死锁有关的某条语句。会话B仍然锁定着表B中的行，而会话A还在耐心地等待这一行可用。收到死锁消息后，会话B必须决定将表B上未执行的工作提交还是回滚，或者继续走另一条路，以后再提交。一旦这个会话执行提交或回滚，另一个阻塞的会话就会继续，好像什么也没有发生过一样。

一、模拟一：两个表之间不同顺序相互更新操作引起的死锁

1、创建两个简单的表A和B，每个表中仅仅包含一个字段id。

```
[ZFZHLHRDB2:oracle]:/oracle>ORACLE_SID=rac1hr2
[ZFZHLHRDB2:oracle]:/oracle>sqlplus / as sysdba

SQL*Plus: Release 11.2.0.4.0 Production on Mon Jun 20 09:40:24 2016

Copyright (c) 1982, 2013, Oracle. All rights reserved.

Connected to:

Oracle Database 11g Enterprise Edition Release 11.2.0.4.0 - 64bit Production
With the Partitioning, Real Application Clusters, Automatic Storage Management, OLAP,
Data Mining and Real Application Testing options

SYS@rac1hr2> select * from v$version;

BANNER
-----
Oracle Database 11g Enterprise Edition Release 11.2.0.4.0 - 64bit Production
PL/SQL Release 11.2.0.4.0 - Production
CORE      11.2.0.4.0      Production
TNS for IBM/AIX RISC System/6000: Version 11.2.0.4.0 - Production
NLSRTL Version 11.2.0.4.0 - Production

SYS@rac1hr2> show parameter cluster

NAME                                TYPE                                VALUE
-----                                -                                -
cluster_database                     boolean                             TRUE
cluster_database_instances           integer                             2
cluster_interconnects                string

SYS@rac1hr2> create table A (id int);

Table created.

SYS@rac1hr2> create table B (id int);

Table created.
```

2、每张表中仅初始化一条数据

```
SYS@rac1hr2> insert into A values (1);
```

```
1 row created.

SYS@rac1hr2> insert into B values (2);

1 row created.

SYS@rac1hr2> commit;

Commit complete.
```

3、在第一个会话session1中更新表A中的记录“1”为“10000”，**不提交**；在第二个会话session2中更新表B中的记录“2”为“20000”，**不提交**
session1的情况：

```
SYS@rac1hr2> insert into A values (1);

1 row created.

SYS@rac1hr2> insert into B values (2);

1 row created.

SYS@rac1hr2> commit;

Commit complete.
```

```
SYS@rac1hr2> SELECT a.SID,
2          b.SERIAL# ,
3          c.SPID
4  FROM    v$mystat a,
5          v$session b ,
6          v$process c
7  WHERE   a.SID = b.SID
8  and b.PADDR=c.ADDR
9  AND     rownum = 1;

SID      SERIAL# SPID
-----
133          3 20906088

SYS@rac1hr2> update A set id = 10000 where id = 1;

1 row updated.
```

session2的情况：

```
SYS@rac1hr2> SELECT a.SID,
2          b.SERIAL# ,
3          c.SPID
4  FROM    v$mystat a,
5          v$session b ,
6          v$process c
7  WHERE   a.SID = b.SID
8  and b.PADDR=c.ADDR
9  AND     rownum = 1;
```

```
SID      SERIAL#  SPID
-----
195      21     11010172

SYS@rac1hr2> update B set id = 20000 where id = 2;

1 row updated.

SYS@rac1hr2>
```

4. 此时，没有任何问题发生。OK，现在注意一下下面的现象，我们再回到会话session1中，更新表B的记录，此时出现了会话阻塞，更新hang住不能继续。

```
SYS@rac1hr2> update B set id = 10000 where id = 2;
```

这里出现了“锁等待”（“阻塞”）的现象，原因很简单，因为在session2中已经对这条数据执行过update操作没有提交表示已经对该行加了行级锁。

```
SYS@rac1hr2> set line 9999
SYS@rac1hr2> SELECT A.INST_ID,
2      A.SID,
3      A.SERIAL#,
4      A.SQL_ID,
5      A.BLOCKING_INSTANCE,
6      A.BLOCKING_SESSION,
7      A.EVENT
8  FROM gv$session a
9  WHERE a.USERNAME IS NOT NULL
10     and a.STATUS = 'ACTIVE'
11     and a.BLOCKING_SESSION IS NOT NULL ;

INST_ID      SID      SERIAL#  SQL_ID      BLOCKING_INSTANCE BLOCKING_SESSION EVENT
-----
2            133      3 6k793mj0duubw      2            195 enq: TX - row lock contention
```

我们可以通过v\$session视图看到，实例2的195阻塞了实例2的133会话，即本实验中的session2阻塞了session1。
6、接下来再执行一条SQL后死锁就会产生：在session2中，更新表A的记录

```
SYS@rac1hr2> update A set id = 10000 where id = 1;
```

这里还是长时间的等待，但是这里发生了死锁，这个时候我们去第一个会话session1中看一下，原先一直在等待的SQL语句报了如下的错误：

```
SYS@rac1hr2> update B set id = 10000 where id = 2;
update B set id = 10000 where id = 2
*
ERROR at line 1:
ORA-00060: deadlock detected while waiting for resource
```

若此时查询v\$session视图可以看到实例2的133阻塞了实例2的195会话，即本实验中的session1阻塞了session2，和刚刚的阻塞情况相反，说明oracle做了自动处理：

```
SYS@rac1hr2> set line 9999
SYS@rac1hr2> SELECT A.INST_ID,
2      A.SID,
3      A.SERIAL#,
4      A.SQL_ID,
5      A.BLOCKING_INSTANCE,
6      A.BLOCKING_SESSION,
7      A.EVENT
8  FROM gv$session a
9  WHERE a.USERNAME IS NOT NULL
10     and a.STATUS = 'ACTIVE'
```

11 and a.BLOCKING_SESSION IS NOT NULL ;

INST_ID	SID	SERIAL#	SQL_ID	BLOCKING_INSTANCE	BLOCKING_SESSION	EVENT

2	195	21	5q7t3877fdu3n	2	133	enq: TX - row lock contention

更进一步：查看一下alert警告日志文件发现有如下的记录：

Mon Jun 20 10:10:56 2016
Global Enqueue Services Deadlock detected. More info in file
/u01/app/oracle/diag/rdbms/rac1hr/rac1hr2/trace/rac1hr2_lmd0_19923170.trc.

若是单机环境，报警日志为：

ORA-00060: Deadlock detected. More info in file
/oracle/app/oracle/diag/rdbms/dlhr/dlhr/trace/dlhr_ora_16973880.trc.



可以看到该文件是由lmd进程生成的，为rac的特有进程，完成CacheFusion的作用，再进一步：看看系统自动生成的trace文件中记录了什么：

```
user session for deadlock lock 0x70001001569c378

sid: 133 ser: 3 audsid: 4294967295 user: 0/SYS

  flags: (0x41) USR/- flags_id1: (0x1) BSY/-/-/-/-
  flags2: (0x40009) -/-/INC

pid: 60 O/S info: user: oracle, term: UNKNOWN, ospid: 20906088

  image: oracle@ZFZHLHRDB2 (TNS V1-V3)

client details:

  O/S info: user: oracle, term: pts/0, ospid: 16122014

  machine: ZFZHLHRDB2 program: sqlplus@ZFZHLHRDB2 (TNS V1-V3)

  application name: sqlplus@ZFZHLHRDB2 (TNS V1-V3), hash value=346557658

current SQL:

update B set id = 10000 where id = 2

DUMP LOCAL BLOCKER: initiate state dump for DEADLOCK

  possible owner[60.20906088] on resource TX-00140013-0000072D


user session for deadlock lock 0x700010015138660

sid: 195 ser: 21 audsid: 4294967295 user: 0/SYS

  flags: (0x41) USR/- flags_id1: (0x1) BSY/-/-/-/-
  flags2: (0x40009) -/-/INC

pid: 46 O/S info: user: oracle, term: UNKNOWN, ospid: 11010172

  image: oracle@ZFZHLHRDB2 (TNS V1-V3)

client details:

  O/S info: user: oracle, term: pts/1, ospid: 16646154

  machine: ZFZHLHRDB2 program: sqlplus@ZFZHLHRDB2 (TNS V1-V3)

  application name: sqlplus@ZFZHLHRDB2 (TNS V1-V3), hash value=346557658

current SQL:

update A set id = 10000 where id = 1

DUMP LOCAL BLOCKER: initiate state dump for DEADLOCK

  possible owner[46.11010172] on resource TX-000B0018-00000416
```

若是单机环境比较明显：

[Transaction Deadlock]

The following deadlock is not an ORACLE error. It is a
deadlock due to user error in the design of an application

```
or from issuing incorrect ad-hoc SQL. The following
information may aid in determining the deadlock:

Deadlock graph:

          -----Blocker(s)-----  -----Waiter(s)-----
Resource Name      process session holds waits  process session holds waits
TX-000d0005-00000047      41      37      X              25      34              X
TX-0008000a-0000036b      25      34      X              41      37              X

session 37: DID 0001-0029-00000003session 34: DID 0001-0019-0000000D
session 34: DID 0001-0019-0000000Dsession 37: DID 0001-0029-00000003

Rows waited on:

  Session 37: obj - rowid = 00015FE7 - AAV/nAABAAAXeBAAA
  (dictionary objn - 90087, file - 1, block - 96129, slot - 0)

  Session 34: obj - rowid = 00015FE6 - AAV/mAABAAAXZ5AAA
  (dictionary objn - 90086, file - 1, block - 95865, slot - 0)
```

注意trace文件中的一行如下提示信息，说明一般情况下都是应用和人为了，和Oracle同学没有关系：
The following deadlock is not an ORACLE error. It is a
deadlock due to user error in the design of an application
or from issuing incorrect ad-hoc SQL.

二、 模拟二：同一张表删除和更新之间引起的死锁

造成死锁的原因就是多个线程或进程对同一个资源的争抢或相互依赖。这里列举一个对同一个资源的争抢造成死锁的实例。

```
SYS@lhrdb S1> CREATE TABLE T_DEADLOCK_DU_LHR ( ID NUMBER, test VARCHAR(10) ) ;

Table created.

SYS@lhrdb S1> INSERT INTO T_DEADLOCK_DU_LHR VALUES(1,'test1');

1 row created.

SYS@lhrdb S1> INSERT INTO T_DEADLOCK_DU_LHR VALUES(2,'test2');

1 row created.

SYS@lhrdb S1> COMMIT;

Commit complete.

SYS@lhrdb S1> SELECT * FROM T_DEADLOCK_DU_LHR;

      ID TEST
-----
1 test1
2 test2
```

会话1更新第一条记录：

```
SYS@lhrdb S1> UPDATE T_DEADLOCK_DU_LHR T SET T.TEST=100 WHERE ID=1;

1 row updated.
```

会话2删除第二条记录：

```
SYS@lhrdb S2> DELETE FROM T_DEADLOCK_DU_LHR T WHERE ID=2;
```

1 row deleted.

接下来会话1更新第二条记录，这就产生了阻塞：

SYS@lhrdb S1> UPDATE T_DEADLOCK_DU_LHR T SET T.TEST=100 WHERE ID=2;

====>>>> 产生了阻塞，会话2阻塞了会话1

SQL_ID	BLOCKING_SESSION	SID	SERIAL#	LOGON_TIME	EVENT
1 c9mg6nd4xkpdv	191	156	5	2016/9/22 9:10:28	enq: TX - row lock contention ...
2 5sbjgfv3mh8zf		191	313	2016/9/22 15:44:32	SQL*Net message from client ...

会话2删除第一条记录：

SYS@lhrdb S2> DELETE FROM T_DEADLOCK_DU_LHR T WHERE ID=1;

====>>>> 产生了阻塞，此时会话1产生死锁

查看会话1：

SYS@lhrdb S1> UPDATE T_DEADLOCK_DU_LHR T SET T.TEST=100 WHERE ID=2;

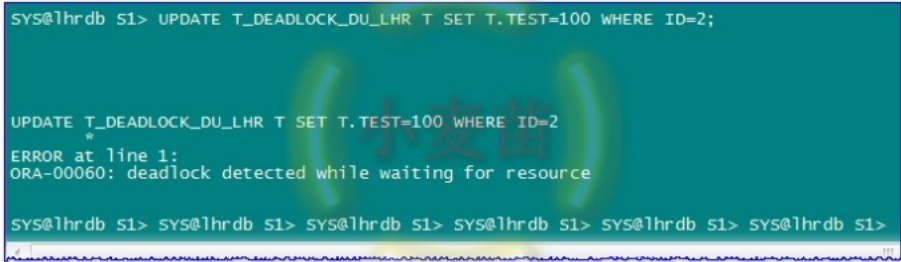
UPDATE T_DEADLOCK_DU_LHR T SET T.TEST=100 WHERE ID=2

*

ERROR at line 1:

ORA-00060: deadlock detected while waiting for resource

SYS@lhrdb S1> SYS@lhrdb S1> SYS@lhrdb S1> SYS@lhrdb S1> SYS@lhrdb S1> SYS@lhrdb S1>



告警日志：

Fri Sep 23 15:10:55 2016

ORA-00060: Deadlock detected. More info in file
/oracle/app/oracle/diag/rdbms/lhrdb/lhrdb/trace/lhrdb_ora_19726460.trc.

内容：

*** 2016-09-23 15:10:55.326
DEADLOCK DETECTED (ORA-00060)

[Transaction Deadlock]

The following deadlock is not an ORACLE error. It is a

```
deadlock due to user error in the design of an application
or from issuing incorrect ad-hoc SQL. The following
information may aid in determining the deadlock:

Deadlock graph:

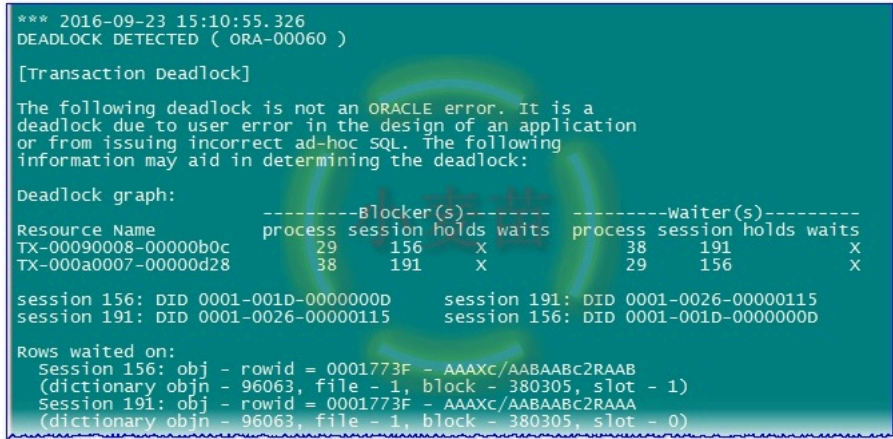
          -----Blocker(s)-----          -----Waiter(s)-----
Resource Name      process session holds waits  process session holds waits
TX-00090008-00000b0c      29      156      X           38      191           X
TX-000a0007-00000d28      38      191      X           29      156           X

session 156: DID 0001-001D-0000000D      session 191: DID 0001-0026-00000115
session 191: DID 0001-0026-00000115      session 156: DID 0001-001D-0000000D

Rows waited on:

Session 156: obj - rowid = 0001773F - AAAXc/AABAABc2RAAB
(dictionary objn - 96063, file - 1, block - 380305, slot - 1)

Session 191: obj - rowid = 0001773F - AAAXc/AABAABc2RAAA
(dictionary objn - 96063, file - 1, block - 380305, slot - 0)
```



2.1.5 自治事务引发的死锁

一般来说构成死锁至少需要两个会话，而自治事务是一个会话可能引发死锁。
自治事务死锁情景:存储过程INSERT表A,然后INSERT表B;其中INSERT表A触发TRIGGER T,T也INSERT表B,T是自治事务(AT),AT试图获取对B的锁,结果B已经被主事务所HOLD,这里会报出来ORA-00060 - 等待资源时检查到死锁。
解决方法:去掉了T中的PRAGMA AUTONOMOUS_TRANSACTION声明,保持和存储过程事务一致。

一、 模拟一：更新

在主事务中如果更新了部分记录，这时若自治事务更新同样的记录，就会造成死锁，下面通过一个简单的例子模拟了这个错误的产生：

```
SYS@lhrdb S1> CREATE TABLE T_DEADLOCK_AT2_LHR(ID NUMBER, NAME VARCHAR2(30));

Table created.

SYS@lhrdb S1> INSERT INTO T_DEADLOCK_AT2_LHR SELECT ROWNUM, TNAME FROM TAB WHERE ROWNUM<=4;

4 rows created.

SYS@lhrdb S1> COMMIT;

Commit complete.

SYS@lhrdb S1> CREATE OR REPLACE PROCEDURE PRO_TESTAT_LHI
2 PRAGMA AUTONOMOUS_TRANSACTION;
```

```
3 BEGIN
4     UPDATE T_DEADLOCK_AT2_LHR SET NAME = NAME WHERE ID = 1;
5     COMMIT;
6 END;
7 /

Procedure created.

SYS@lhrdb S1> UPDATE T_DEADLOCK_AT2_LHR SET NAME = NAME WHERE ID = 1;

1 row updated.

SYS@lhrdb S1> EXEC PRO_TESTAT_LHR;

BEGIN PRO_TESTAT_LHR; END;

*
ERROR at line 1:
ORA-00060: deadlock detected while waiting for resource
ORA-06512: at "SYS.PRO_TESTAT_LHR", line 4
ORA-06512: at line 1

SYS@lhrdb S1> SYS@lhrdb S1>
```

在使用自治事务的时候要避免当前事务锁定的记录和自治事务中锁定的记录相互冲突。
告警日志:

```
Fri Sep 23 14:03:10 2016

ORA-00060: Deadlock detected. More info in file
/oracle/app/oracle/diag/rdbms/lhrdb/lhrdb/trace/lhrdb_ora_19726460.trc.
```

内容:

```
*** 2016-09-23 14:10:34.974

DEADLOCK DETECTED ( ORA-00060 )

[Transaction Deadlock]

The following deadlock is not an ORACLE error. It is a
deadlock due to user error in the design of an application
or from issuing incorrect ad-hoc SQL. The following
information may aid in determining the deadlock:

Deadlock graph:

          -----Blocker(s)-----  -----Waiter(s)-----
Resource Name      process session holds waits  process session holds waits
TX-000a0015-00000d25      38      191      X              38      191      X

session 191: DID 0001-0026-00000115      session 191: DID 0001-0026-00000115

Rows waited on:

  Session 191: obj - rowid = 0001773A - AAAXc6AABAABc2BAAA
  (dictionary objn - 96058, file - 1, block - 380289, slot - 0)

----- Information for the OTHER waiting sessions -----
----- End of information for the OTHER waiting sessions -----
```

```
Information for THIS session:
```

```
----- Current SQL Statement for this session (sql_id=3w3thujdh1y3a) -----
```

```
UPDATE T_DEADLOCK_AT2_LHR SET NAME = NAME WHERE ID = 1
```

```
----- PL/SQL Stack -----
```

```
----- PL/SQL Call Stack -----
```

object	line	object
handle	number	name
700010052c9cf40	4	procedure SYS.PRO_TESTAT_LHR
700010052cb7588	1	anonymous block

```
*** 2016-09-23 14:10:34.974
DEADLOCK DETECTED ( ORA-00060 )

[Transaction Deadlock]

The following deadlock is not an ORACLE error. It is a
deadlock due to user error in the design of an application
or from issuing incorrect ad-hoc SQL. The following
information may aid in determining the deadlock:

Deadlock graph:
-----Blocker(s)-----
Resource Name      process session holds waits
TX-000a0015-00000d25 38      191      X
session 191: DID 0001-0026-00000115
-----waiter(s)-----
process session holds waits
38      191      X
session 191: DID 0001-0026-00000115

Rows waited on:
  session 191: obj - rowid = 0001773A - AAAXc6AABAAbc2BAAA
  (dictionary objn - 96058, file - 1, block - 380289, slot - 0)

----- Information for the OTHER waiting sessions -----
----- End of information for the OTHER waiting sessions -----

Information for THIS session:

----- Current SQL Statement for this session (sql_id=3w3thujdh1y3a) -----
UPDATE T_DEADLOCK_AT2_LHR SET NAME = NAME WHERE ID = 1
----- PL/SQL Stack -----
----- PL/SQL Call Stack -----
  object      line  object
  handle      number name
700010052c9cf40      4  procedure SYS.PRO_TESTAT_LHR
700010052cb7588      1  anonymous block
=====
PROCESS STATE
```

1、脚本

```
CREATE TABLE T_DEADLOCK_AT2_LHR(ID NUMBER, NAME VARCHAR2(30));

INSERT INTO T_DEADLOCK_AT2_LHR SELECT ROWNUM, TNAME FROM TAB WHERE ROWNUM<=4;

COMMIT;

CREATE OR REPLACE PROCEDURE PRO_TESTAT_LHR AS

  PRAGMA AUTONOMOUS_TRANSACTION;

BEGIN

  UPDATE T_DEADLOCK_AT2_LHR SET NAME = NAME WHERE ID = 1;

  COMMIT;

END;

/

UPDATE T_DEADLOCK_AT2_LHR SET NAME = NAME WHERE ID = 1;

exec PRO_TESTAT_LHR;

ERROR AT LINE 1:

ORA-00060: DEADLOCK DETECTED WHILE WAITING FOR RESOURCE

ORA-06512: AT "SYS.PRO_TESTAT_LHR", LINE 4

ORA-06512: AT LINE 3
```

二、 模拟二：插入

主事务和自治事务插入的是同一个主键值也会引起死锁。

```
SYS@lhrdb S1> CREATE TABLE T_DEADLOCK_AT_LHR(X INT PRIMARY KEY,Y INT);

Table created.

SYS@lhrdb S1> CREATE OR REPLACE PROCEDURE PRO_AUTO_PROC_LHR(P_X INT,P_Y INT) AS
  2 PRAGMA AUTONOMOUS_TRANSACTION;
  3 BEGIN
  4   INSERT INTO T_DEADLOCK_AT_LHR(X,Y) VALUES(P_X,P_Y);
  5   COMMIT;
  6 END;
  7 /

Procedure created.

SYS@lhrdb S1> INSERT INTO T_DEADLOCK_AT_LHR(X,Y) VALUES(1,1);

1 row created.

SYS@lhrdb S1> EXEC PRO_AUTO_PROC_LHR(1,2);
BEGIN PRO_AUTO_PROC_LHR(1,2); END;

*
ERROR at line 1:
ORA-00060: deadlock detected while waiting for resource
ORA-06512: at "SYS.PRO_AUTO_PROC_LHR", line 4
ORA-06512: at line 1
```

告警日志:

```
Fri Sep 23 13:49:06 2016

ORA-00060: Deadlock detected. More info in file
/oracle/app/oracle/diag/rdbms/lhrdb/lhrdb/trace/lhrdb_ora_19726460.trc.
```

内容:

```
*** 2016-09-23 13:49:06.546

DEADLOCK DETECTED ( ORA-00060 )

[Transaction Deadlock]

The following deadlock is not an ORACLE error. It is a
deadlock due to user error in the design of an application
or from issuing incorrect ad-hoc SQL. The following
information may aid in determining the deadlock:

Deadlock graph:
          -----Blocker(s)-----  -----Waiter(s)-----
Resource Name      process session holds waits  process session holds waits
TX-000a000b-00000d28      29      156      X          29      156          S

session 156: DID 0001-001D-0000000D      session 156: DID 0001-001D-0000000D

Rows waited on:
  Session 156: no row
```

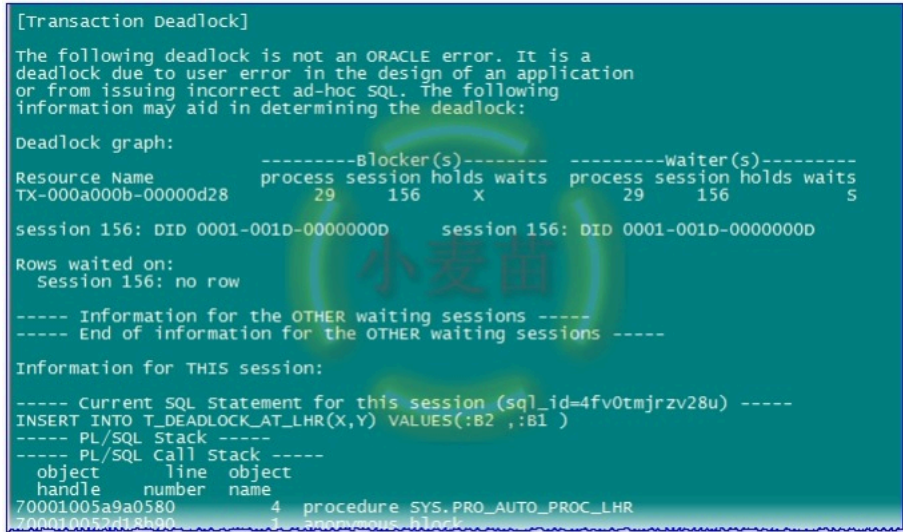
```
----- Information for the OTHER waiting sessions -----
----- End of information for the OTHER waiting sessions -----

Information for THIS session:

----- Current SQL Statement for this session (sql_id=4fv0tmjrzv28u) -----
INSERT INTO T_DEADLOCK_AT_LHR(X,Y) VALUES(:B2, :B1 )

----- PL/SQL Stack -----
----- PL/SQL Call Stack -----

object      line object
handle      number name
70001005a9a0580      4 procedure SYS.PRO_AUTO_PROC_LHR
700010052d18b90      1 anonymous block
```



可以看到，等待的和持有锁的是同一个会话，根据trace信息记录的对象，发现问题是自治事务导致的。

1、脚本

```
CREATE TABLE T_DEADLOCK_AT_LHR(X INT PRIMARY KEY,Y INT);

CREATE OR REPLACE PROCEDURE PRO_AUTO_PROC_LHR(P_X INT,P_Y INT) AS
PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
    INSERT INTO T_DEADLOCK_AT_LHR(X,Y) VALUES (P_X,P_Y);
    COMMIT;
END;
/

INSERT INTO T_DEADLOCK_AT_LHR(X,Y) VALUES (1,1);

SQL> EXEC PRO_AUTO_PROC_LHR(1,2);
BEGIN AUTO_PROC(1,2); END;

*

ERROR AT LINE 1:
ORA-00060: DEADLOCK DETECTED WHILE WAITING FOR RESOURCE
ORA-06512: AT "BOCNET.AUTO_PROC", LINE 4
```

ORA-06512: AT LINE 1

2.2 块级死锁

块级死锁其实指的就是ITL死锁。

2.2.1 ITL简介

ITL (Interested Transaction List) 是Oracle数据块内部的一个组成部分，用来记录该块所有发生的事务，有的时候也叫ITL槽位。如果一个事务一直没有提交，那么，这个事务将一直占用一个ITL槽位，ITL里面记录了事务信息、回滚段的入口和事务类型等等。如果这个事务已经提交，那么，ITL槽位中还保存有这个事务提交时候的SCN号。ITL的个数受表的存储参数INITRANS控制，在一个块内部，默认分配了2个ITL的个数，如果这个块内还有空闲空间，那么Oracle是可以利用这些空闲空间再分配ITL。如果没有了空闲空间，那么，这个块因为不能分配新的ITL，所以，就可能发生ITL等待。如果在并发量特别大的系统中，那么最好分配足够的ITL个数，或者设置足够的PCTFREE，保证ITL能扩展，但是PCTFREE有可能是被行数据给消耗掉的，例如UPDATE，所以，也有可能导致块内部的空间不够而导致ITL等待，出现了ITL等待就可能发生ITL死锁。

一、ITL结构

如果DUMP一个块（命令：alter system dump datafile X block XXX;），那么在DUMP文件中就可以看到ITL信息：

Itl	Xid	Uba	Flag	Lck	Scn/Fsc
0x01	0x0008.002.000009e9	0x00c0108b.04ac.24	--U-	3	fsc 0x0000.00752951
0x02	0x0000.000.00000000	0x00000000.0000.00	----	0	fsc 0x0000.00000000

Itl	Xid	Uba	Flag	Lck	Scn/Fsc
0x01	0x0008.002.000009e9	0x00c0108b.04ac.24	--U-	3	fsc 0x0000.00752951
0x02	0x0000.000.00000000	0x00000000.0000.00	----	0	fsc 0x0000.00000000

- 1) Itl: ITL事务槽编号，ITL事务槽号的流水编号
- 2) Xid: 代表对应的事务id (transac[X]tion identified)，在回滚段事务表中有一条记录和这个事务对应。Xid由三列使用十六进制编码的数字列表示，分别是：Undo Segment Number +Transaction Table Slot Number+ Wrap,即由undo段号+undo槽号+undo槽号的覆盖次数三部分组成，即usn.slot.sqn，这里0x0008.002.000009e9转换为10进制为8.2.2537，从下边的查询出的结果是相对应的。

```
SYS@lhrdb> select xidusn,xidslot,xidsqn,ubafil,ubablk,ubasqn,ubarec from v$transaction;
```

XIDUSN	XIDSLLOT	XIDSQN	UBAFIL	UBABLK	UBASQN	UBAREC
8	2	2537	3	4235	1196	36

- 3) Uba: (Undo Block Address)，该事务对应的回滚段地址，记录了最近一次的该记录的前镜像（修改前的值）。Uba组成：Undo块地址 (undo文件号和数据块号) +回滚序列号+回滚记录号。多版本一致读是Oracle保证读操作不会被事务阻塞的重要特性。当Server Process需要查询一个正在被事务修改，但是尚未提交的数据时，就根据ITL上的uba定位到对应Undo前镜像数据位置。这里的Uba为：0x00c0108b.04ac.24，其中00c0108b (16进制) =0000 0000 1100 0000 0001 0000 1000 1011 (2进制，共32位，前10位代表文件号，后22位代表数据块号) =文件号为3，块号为4235: (10进制)；04ac (16进制) =1196 (10进制)；24 (16进制) =36 (10进制)。这个结果和v\$transaction查询出来的结果一致。

SELECT UBAFIL 回滚段文件号,UBABLK 数据块号,UBASQN 回滚序列号,UBAREC 回滚记录号 FROM v\$transaction ; --查看UBA

- 4) Flag: 事务标志位，即当前事务槽的状态信息。这个标志位就记录了这个事务的操作状态，各个标志的含义分别是：

标识	简介
----	事务是活动的，未提交，或者在块清除前提交事务
C---	transaction has been committed and locks cleaned out - -事务已经提交，锁已经被清除（提交）
-B--	this undo record contains the undo for this ITL entry
--U-	transaction committed (maybe long ago); SCN is an upper bound --事务已经提交，但是锁还没有清除（快速提交）
---T	transaction was still active at block cleanout SCN -- 块清除的scn被记录时，该事务仍然是活动的，块上如果有已经提交的事务，那么在clean ount的时候，块会被进行清除，但是这个块里面的事务不会被清除。
C-U-	块被延迟清除，回滚段的信息已经改写，SCN显示为最小的SCN，需要由回滚段重新生成，例如在提交以前，数据块已经flush到数据文件上。

- 5) Lck: 表示这个事务所影响的行数，锁住了几行数据，对应有几个行锁。我们看到01号事物槽Lck为3，因为该事物槽中的事物Flag为U，证明该事物已经提交，但是锁还没有清除。再比如对于下边这个ITL：

Itl	Xid	Uba	Flag	Lck	Scn/Fsc
0x01	0x0006.020.00000271	0x00800205.0257.13	C---		1732c4
0x02	0x0008.006.00000279	0x00800351.0278.15	----		J000000

我们看到01号事物槽Lck为0，因为该事物槽中的事物Flag为C，证明该事物已经提交，锁也被清楚掉了，该事物槽可以被重用了。02号事物槽Lck为1，是因为我对第一行做了一个更新，并且没有提交，Flag为“----”说明该事物是活动的。

6) **Scn/Fsc**: Commit SCN或者快速提交 (Fast Commit Fsc) 的SCN。Scn=SCN of committed TX; Fsc=Free space credit (bytes) 每条记录中的行级锁对应Itl条目lb，对应于Itl列表中的序号，即那个事务在该记录上产生的锁。一个事物只有在提交之后才会在ITL事物槽中记录SCN。

二、ITL个数

ITL的个数，受参数INITRANS控制，最大ITL个数受MAXTRANS控制（11g已废弃MAXTRANS），在一个块内部，默认分配了2个ITL的个数。ITL是block级的概念，一个ITL占用块46B的空间，参数INITRANS意味着块中除去block header外一部分存储空间无法被记录使用（46B*INITRANS），当块中还有一定的FREE SPACE时，ORACLE可以使用FREE SPACE构建ITL供事务使用，如果这个块内还有空闲空间，那么Oracle是可以利用这些空闲空间并再分配ITL。如果没有了空闲空间（free space），那么，这个块因为不能分配新的ITL，所以就可能发生ITL等待，即enq: TX - allocate ITL entry等待事件。**注意：10g以后MAXTRANS参数被废弃，默认最大支持255个并发。**

如果在并发量特别大的系统中，最好分配足够的ITL个数，其实它并浪费不了太多的空间，或者，设置足够的PCTFREE，保证ITL能扩展，但是PCTFREE有可能是被行数据给消耗掉的，如UPDATE，所以，也有可能导致块内部的空间不够而导致ITL等待。

对于表（数据块）来说，INITRANS这个参数的默认值是1。对于索引（索引块）来说，这个参数默认值是2。

ITL等待表现出的等待事件为“TX - allocate ITL entry”，根据MOS (Troubleshooting waits for 'enq: TX - allocate ITL entry' (Doc ID 1472175.1) 提供的解决办法，需要修改一些参数，SQL如下，这里假设用户名为TLHR，表名为TLHRBOKBAL，表上的索引名为PK_TLHRBOKBAL：

```
ALTER TABLE TLHR.TLHRBOKBAL PCTFREE 20 INITRANS 16;

ALTER TABLE TLHR.TLHRBOKBAL MOVE NOLOGGING PARALLEL 12;

ALTER TABLE TLHR.TLHRBOKBAL LOGGING NOPARALLEL;

ALTER INDEX TLHR.PK_TLHRBOKBAL REBUILD PCTFREE 20 INITRANS 16 NOLOGGING PARALLEL 12;

ALTER INDEX TLHR.PK_TLHRBOKBAL LOGGING NOPARALLEL;
```

无MOS权限的朋友可以去<http://blog.itpub.net/26736162/viewspace-2124531/>阅读。

2.2.2 ITL引起的死锁案例处理

由ITL不足引发的块级死锁的一个处理案例可以参考我的BLOG:<http://blog.itpub.net/26736162/viewspace-2124771/>、<http://blog.itpub.net/26736162/viewspace-2124735/>。

2.2.3 ITL死锁的模拟

我们首先创建一张表T_ITL_LHR，这里指定PCTFREE为0，INITRANS为1，就是为了观察到ITL的真实等待情况，然后我们给这些块内插入数据，把块填满，让它不能有空间分配。

```
SYS@lhrdb21> SELECT * FROM V$VERSION;

BANNER
-----
Oracle Database 11g Enterprise Edition Release 11.2.0.4.0 - 64bit Production
PL/SQL Release 11.2.0.4.0 - Production
CORE      11.2.0.4.0      Production
TNS for IBM/AIX RISC System/6000: Version 11.2.0.4.0 - Production
NLSRTL Version 11.2.0.4.0 - Production

SYS@lhrdb21> SHOW PARAMETER CLUSTER

NAME                                TYPE          VALUE
-----
cluster_database                     boolean       TRUE
cluster_database_instances           integer       2
cluster_interconnects                string

SYS@lhrdb21> CREATE TABLE T_ITL_LHR(A INT) PCTFREE 0 INITRANS 1;

Table created.

SYS@lhrdb21> BEGIN

  2   FOR I IN 1 .. 2000 LOOP

  3       INSERT INTO T_ITL_LHR VALUES (I);

  4   END LOOP;

  5 END;
```

```
6 /

PL/SQL procedure successfully completed.

SYS@lhrdb21> COMMIT;

Commit complete.
```

我们检查数据填充的情况：

```
SYS@lhrdb21> SELECT F, B, COUNT(*)
2      FROM (SELECT DBMS_ROWID.ROWID_RELATIVE_FNO(ROWID) F,
3                  DBMS_ROWID.ROWID_BLOCK_NUMBER(ROWID) B
4                  FROM T_ITL_LHR)
5 GROUP BY F, B
6 ORDER BY F,B;
```

F	B	COUNT(*)
1	94953	734
1	94954	734
1	94955	532

可以发现，这2000条数据分布在3个块内部，其中有2个块（94953和94954）填满了，一个块（94955）是半满的。因为有2个ITL槽位，我们需要拿2个满的数据块，4个进程来模拟ITL死锁：

实验步骤	会话	SID	要更新的块号	要更新的行号	是否有阻塞
步骤一	1	19	94953	1	N
	2	79	94953	2	N
	3	78	94954	1	N
	4	139	94954	2	N

会话1：

```
SYS@lhrdb21> SELECT USERENV('SID') FROM DUAL;

USERENV('SID')
-----
19

SYS@lhrdb21> UPDATE T_ITL_LHR SET A=A
2 WHERE DBMS_ROWID.ROWID_BLOCK_NUMBER(ROWID)=94953
3 AND DBMS_ROWID.ROWID_ROW_NUMBER(ROWID)=1;

1 row updated.
```

会话2：

```
SYS@lhrdb21> SELECT USERENV('SID') FROM DUAL;

USERENV('SID')
-----
79

SYS@lhrdb21> UPDATE T_ITL_LHR SET A=A
2 WHERE DBMS_ROWID.ROWID_BLOCK_NUMBER(ROWID)=94953
3 AND DBMS_ROWID.ROWID_ROW_NUMBER(ROWID)=2;
```

1 row updated.

会话3:

```
SYS@lhrdb21> SELECT USERENV('SID') FROM DUAL;

USERENV('SID')
-----
78

SYS@lhrdb21> UPDATE T_ITL_LHR SET A=A
2 WHERE DBMS_ROWID.ROWID_BLOCK_NUMBER(ROWID)=94954
3 AND DBMS_ROWID.ROWID_ROW_NUMBER(ROWID)=1;

1 row updated.
```

会话4:

```
SYS@lhrdb21> SELECT USERENV('SID') FROM DUAL;

USERENV('SID')
-----
139

SYS@lhrdb21> UPDATE T_ITL_LHR SET A=A
2 WHERE DBMS_ROWID.ROWID_BLOCK_NUMBER(ROWID)=94954
3 AND DBMS_ROWID.ROWID_ROW_NUMBER(ROWID)=2;

1 row updated.
```

```
这个时候系统不存在阻塞,
SELECT NVL(A.SQL_ID, A.PREV_SQL_ID) SQL_ID,
       A.BLOCKING_SESSION,
       A.SID,
       A.SERIAL#,
       A.LOGON_TIME,
       A.EVENT
FROM GV$SESSION A
WHERE A.SID IN (19, 79,78,139)
ORDER BY A.LOGON_TIME;
```

SQL_ID	BLOCKING_SESSION	SID	SERIAL#	LOGON_TIME	EVENT
1 dbqxjrxvp6zuz		19	4933	2016/9/9 17:18:55	SQL*Net message from client ...
2 2ffxtkr2wv6bp		79	16451	2016/9/9 17:19:05	SQL*Net message from client ...
3 75sn2pj2yblfc		78	3461	2016/9/9 17:19:42	SQL*Net message from client ...
4 cmrck75g0mx02		139	3503	2016/9/9 17:20:04	SQL*Net message from client ...

以上4个进程把2个不同块的4个ITL槽位给消耗光了，现在的情况，就是让他们互相锁住，达成死锁条件，回到会话1，更新块94954，注意，以上4个操作，包括以下的操作，更新的根本不是同一行数据，主要是为了防止出现的是行锁等待。

实验步骤	会话	SID	要更新的块号	要更新的行号	是否有阻塞
步骤一	1	19	94953	1	N
	2	79	94953	2	N
	3	78	94954	1	N
	4	139	94954	2	N
步骤二	1	19	94954	3	
	3	78	94953	3	Y

会话1：

```
UPDATE T_ITL_LHR SET A=A
WHERE DBMS_ROWID.ROWID_BLOCK_NUMBER(ROWID)=94954
AND DBMS_ROWID.ROWID_ROW_NUMBER(ROWID)=3;
```

```
SYS@1hrdb21> UPDATE T_ITL_LHR SET A=A
2  WHERE DBMS_ROWID.ROWID_BLOCK_NUMBER(ROWID)=94954
3  AND DBMS_ROWID.ROWID_ROW_NUMBER(ROWID)=3;
```

会话1出现了等待。

会话3：

```
UPDATE T_ITL_LHR SET A=A
WHERE DBMS_ROWID.ROWID_BLOCK_NUMBER(ROWID)=94953
AND DBMS_ROWID.ROWID_ROW_NUMBER(ROWID)=3;
```

```
SYS@1hrdb21> UPDATE T_ITL_LHR SET A=A
2  WHERE DBMS_ROWID.ROWID_BLOCK_NUMBER(ROWID)=94953
3  AND DBMS_ROWID.ROWID_ROW_NUMBER(ROWID)=3;
```

会话3发现出现了等待。

我们查询阻塞的具体情况：

```
SELECT NVL(A.SQL_ID, A.PREV_SQL_ID) SQL_ID,
       A.BLOCKING_SESSION,
       A.SID,
       A.SERIAL#,
       A.LOGON_TIME,
       A.EVENT
FROM GV$SESSION A
WHERE A.SID IN (19, 79, 78, 139)
ORDER BY A.LOGON_TIME;
```

SQL_ID	BLOCKING_SESSION	SID	SERIAL#	LOGON_TIME	EVENT
1 dptxyampd11y1	139	19	4933	2016/9/9 17:18:55	enq: TX - allocate ITL entry
2 2ffxtkr2wv6bp		79	16451	2016/9/9 17:19:05	SQL*Net message from client
3 202gq918npxrq	79	78	3461	2016/9/9 17:19:42	enq: TX - allocate ITL entry
4 cmrck75g0mx02		139	3503	2016/9/9 17:20:04	SQL*Net message from client

可以看到，会话1被会话4阻塞了，会话3被会话2阻塞了。

注意，如果是9i，在这里就报死锁了，但是在10g里面，这个时候，死锁是不会发生的，因为这里的会话1还可以等待会话4释放资源，会话3还可以等待会话2释放资源，只要会话2与会话4释放了资源，整个环境又活了，那么我们需要把这两个进程也塞住。

出现的是行锁等待。

实验步骤	会话	SID	要更新的块号	要更新的行号	是否有阻塞
步骤一	1	19	94953	1	N
	2	79	94953	2	N
	3	78	94954	1	N
	4	139	94954	2	N
步骤二	1	19	94954	3	Y
	3	78	94953	3	Y
步骤三	2	79	94954	4	Y
	4	139	94953	4	Y

会话2，注意，我们也不是更新的同一行数据：

```
UPDATE T_ITL_LHR SET A=A
WHERE DBMS_ROWID.ROWID_BLOCK_NUMBER(ROWID)=94954
AND DBMS_ROWID.ROWID_ROW_NUMBER(ROWID)=4;
```

```
SYS@1hrdb21> UPDATE T_ITL_LHR SET A=A
2  WHERE DBMS_ROWID.ROWID_BLOCK_NUMBER(ROWID)=94954
3  AND DBMS_ROWID.ROWID_ROW_NUMBER(ROWID)=4;
```

会话2出现了等待，具体阻塞情况：

SQL_ID	BLOCKING_SESSION	SID	SERIAL#	LOGON_TIME	EVENT
dptxyampd11y1	78	19	4933	2016/9/9 17:18:55	enq: TX - allocate ITL entry ...
gs5c8j35k6j51	139	79	16451	2016/9/9 17:19:05	enq: TX - allocate ITL entry ...
202gv918npxrq	79	78	3461	2016/9/9 17:19:42	enq: TX - allocate ITL entry ...
cmrck75q0mx02		139	3503	2016/9/9 17:20:04	SQL*Net message from client ...

我做了几次实验，会话2执行完SQL后，会话3到这里就报出了死锁，但有的时候并没有产生死锁，应该跟系统的阻塞顺序有关，若没有产生死锁，我们可以继续会话4的操作。

```
SYS@lhrdb21> UPDATE T_ITL_LHR SET A=A
2 WHERE DBMS_ROWID.ROWID_BLOCK_NUMBER(ROWID)=94953
3 AND DBMS_ROWID.ROWID_ROW_NUMBER(ROWID)=3;

UPDATE T_ITL_LHR SET A=A
*
ERROR at line 1:
ORA-00060: deadlock detected while waiting for resource
```

会话4，注意，我们也不是更新的同一行数据：

```
UPDATE T_ITL_LHR SET A=A
WHERE DBMS_ROWID.ROWID_BLOCK_NUMBER(ROWID)=94953
AND DBMS_ROWID.ROWID_ROW_NUMBER(ROWID)=4;

SYS@lhrdb21> UPDATE T_ITL_LHR SET A=A
2 WHERE DBMS_ROWID.ROWID_BLOCK_NUMBER(ROWID)=94953
3 AND DBMS_ROWID.ROWID_ROW_NUMBER(ROWID)=4;
```

会话4发现出现了等待。

SQL_ID	BLOCKING_SESSION	SID	SERIAL#	LOGON_TIME	EVENT
dptxyampd11y1	78	19	4933	2016/9/9 17:18:55	enq: TX - allocate ITL entry ...
gs5c8j35k6j51	78	79	16451	2016/9/9 17:19:05	enq: TX - allocate ITL entry ...
202gv918npxrq	19	78	3461	2016/9/9 17:19:42	enq: TX - allocate ITL entry ...
98buh3cjuc2r8	79	139	3503	2016/9/9 17:20:04	enq: TX - allocate ITL entry ...

虽然，以上的每个更新语句，更新的都不是同一个数据行，但是，的确，所有的进程都被阻塞住了，那么，死锁的条件也达到了，等待一会（这个时间有个隐含参数来控制的：_lm dd interval），我们可以看到，会话2出现提示，死锁：

```
SYS@lhrdb21> UPDATE T_ITL_LHR SET A=A
2 WHERE DBMS_ROWID.ROWID_BLOCK_NUMBER(ROWID)=94954
3 AND DBMS_ROWID.ROWID_ROW_NUMBER(ROWID)=4;

UPDATE T_ITL_LHR SET A=A
*
ERROR at line 1:
ORA-00060: deadlock detected while waiting for resource
```

报出死锁之后的阻塞情况：

SQL_ID	BLOCKING_SESSION	SID	SERIAL#	LOGON_TIME	EVENT
1 dptxyampd11y1	78	19	4933	2016/9/9 17:18:55	enq: TX - allocate ITL entry ...
2 gs5c8j35k6j51	79	79	16451	2016/9/9 17:19:05	SQL*Net message from client ...
3 202gv918npxrq	79	78	3461	2016/9/9 17:19:42	enq: TX - allocate ITL entry ...
4 98buh3cjuc2r8	79	139	3503	2016/9/9 17:20:04	enq: TX - allocate ITL entry ...

我们可以在会话2上继续执行步骤三中的SQL，依然会产生死锁。生成死锁后，在告警日志中有下边的语句：

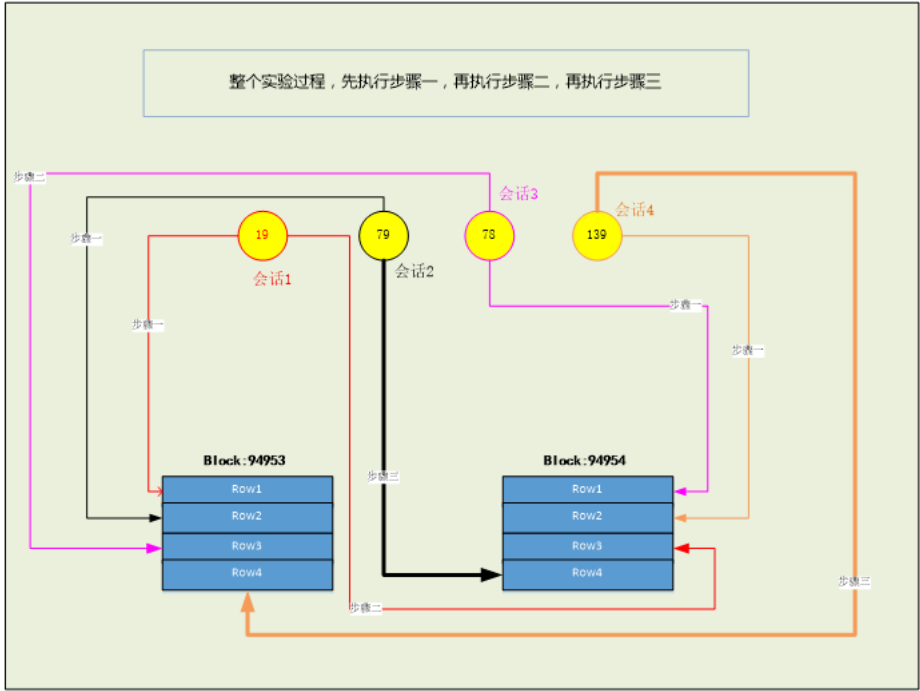
```
Fri Sep 09 17:56:55 2016
Global Enqueue Services Deadlock detected. More info in file
/oracle/app/oracle/diag/rdbms/lhrdb2/lhrdb21/trace/lhrdb21_lmd0_17039368.trc.
```

其中的内容有非常经典的一段Global Wait-For-Graph (WFG)：

```
*** 2016-09-09 17:48:22.216
Submitting asynchronous dump request [1c]. summary=[ges process stack dump (kjdg1blkrdm1)].
Global blockers dump end:-----
Global Wait-For-Graph (WFG) at ddTS[0.395] :
BLOCKED 0x700010063d59b90 3 wq 2 cvtops x1001 TX 0x7000b [?0029-00008387] inst 1
BLOCKER 0x700010063c6d268 3 wq 1 cvtops x28 TX 0x7000b.( 02D-00003742] inst 1
BLOCKED 0x700010063d5adc8 3 wq 2 cvtops x1 TX 0x30021.0x848 (ext 0x2,0x0) [1002-002D-00003742] inst 1
```

```
BLOCKER 0x700010063d5a4b8 3 wq 1 cvtops x28 TX 0x30021.0x848(ext 0x2,0x0)[1002-0029-00008387] inst 1
```

该实验过程可能有点复杂，小麦苗画了个图来说明整个实验过程：



About Me

- 本文作者：小麦苗，只专注于数据库的技术，更注重技术的运用
- 本文在itpub (<http://blog.itpub.net/26736162>)、博客园 (<http://www.cnblogs.com/lhrbest>) 和个人微信公众号 (xiaomaimiaolhr) 上有同步更新
- 本文itpub地址: <http://blog.itpub.net/26736162/viewspace-2127247/>
- 本文博客园地址: <http://www.cnblogs.com/lhrbest/p/6005702.html>
- 本文pdf版及小麦苗网盘地址: <http://blog.itpub.net/26736162/viewspace-1624453/>
- QQ群: 230161599 微信群: 私聊
- 联系我请加QQ好友 (642808185)，注明添加缘由
- 于 2016-09-01 15:00 ~ 2016-10-20 19:00 在中行完成
- 文章内容来源于小麦苗的学习笔记，部分整理自网络，若有侵权或不当之处还请谅解
- 【版权所有，文章允许转载，但须以链接方式注明源地址，否则追究法律责任】

手机长按下图识别二维码或微信客户端扫描下边的二维码来关注小麦苗的微信公众号：xiaomaimiaolhr, 免费学习最实用的数据库技术。



小麦苗

- 小麦苗的微店: <https://weidian.com/s/793741433?wfr=c&iffr=shopdetail>

使用微信客户端扫描下面的二维码来关注小麦苗的微信公众号（DB宝），学习最实用的数据库技术。

[好文要顶](#)[关注我](#)[收藏该文](#)[微信分享](#)

DB宝

粉丝 - 212 关注 - 74

[+加关注](#)

1

0

[升级成为会员](#)[« 上一篇：【函数】Oracle函数系列（1）--字符函数](#)[» 下一篇：【体系结构】Oracle参数介绍](#)

posted @ 2016-10-27 21:36 DB宝

阅读(8983) 评论(2) 编辑 收藏 举报

[会员力量，点亮园子希望](#)[刷新页面](#) [返回顶部](#)

登录后才能查看或发表评论，立即 [登录](#) 或者 [逛逛](#) [博客园首页](#)

- 【推荐】轻量又高性能的 SSH 工具 iShell：AI 加持，快人一步
- 【推荐】100%开源！大型工业跨平台软件C++源码提供，建模，组态！
- 【推荐】2024阿里云超值优品季，精心为您准备的上云首选必备产品
- 【推荐】「废话少说，放码过来」：博客园2024夏季短袖T恤上架啦
- 【推荐】会员力量，点亮园子希望，期待您升级成为博客园VIP会员



编辑推荐：

- 旧物利用 - 将机顶盒改造为一台 Linux 开发机！
- 这是DDD建模最难的部分（其实很简单）
- 为了落地DDD，我是这样“PUA”大家的
- 神秘 Arco 样式出现，祭出 Webpack 解决预期外的引用问题
- c#12 实验特性 Interceptor 如何使用的一个示例

阅读排行：

- 这是DDD建模最难的部分（其实很简单）
- 旧物利用 - 将机顶盒改造为一台Linux开发机！
- 实习第一天，不小心透露了，我是拆迁户
- 为了给Javaer落地DDD，我们不得不写开源组件
- 七天.NET 8操作SQLite入门到实战详细教程（选型、开发、发布、部署）